# Java Basics — Section 2: Basic Programming in Java

## 2.1 Character Set

A character set is the group of symbols a language allows you to write in source code. Java source files use **Unicode**, so you can technically write identifiers and strings in any language. For everyday programming you work with these categories:

| Category | Characters | Where it is used |
|---|---|---|
| Alphabets | `A-Z`, `a-z` (plus any Unicode letter) | Names of variables, classes, methods; keywords |
| Numeric (digits) | `0-9` | Number literals, and inside names (not at the start) |
| Alphanumeric | Letters and digits together, e.g. `user1`, `item42` | Identifiers |
| Special symbols | `+ - * / % = < > ! & \| ^ ~ ? : ; , . ( ) [ ] { } @ # $ _ " '` | Operators, separators, annotations, literals |
| White space | Space, tab, newline (`\n`), carriage return (`\r`), form feed | Separates tokens; ignored by the compiler except inside string literals |

Two points beginners often miss. First, Java is **case-sensitive**: `total`, `Total` and `TOTAL` are three different names. Second, white space is mostly free-form: `int x=5;` and `int x = 5 ;` mean the same thing, but you cannot split a word in the middle (`in t x`).

```
int age = 25;        // 'int', 'age' are alphabets, '25' numeric, '=' and ';'
special symbols
String user1 = "A";  // 'user1' is alphanumeric
```

## 2.2 Escape Sequences

Some characters cannot be typed directly inside a string or char literal (a new line, a double quote that would end the string). An **escape sequence** starts with a backslash `\` followed by a code, and stands for one character.

| Sequence | Meaning | Example and output |
|---|---|---|
| `\n` | New line (line feed) | `"A\nB"` prints A, then B on the next line |
| `\t` | Horizontal tab | `"A\tB"` prints `A` and `B` separated by a tab |
| `\r` | Carriage return (cursor to line start) | `"abc\rX"` on a console shows `Xbc` |
| `\b` | Backspace | `"ab\bc"` shows `ac` on most consoles |
| `\f` | Form feed | Rare; page break on old printers |
| `\s` | Space (Java 15+) | Keeps trailing space in text blocks |
| `\"` | Double quote | `"He said \"Hi\""` prints He said "Hi" |
| `\'` | Single quote | `'\''` is the character `'` |
| `\\` | Backslash itself | `"C:\\temp"` prints C:\temp |
| `\0` to `\377` | Octal character code | `'\101'` is `A` (octal 101 = decimal 65) |
| `\uXXXX` | Unicode code unit, 4 hex digits | `'\u0041'` is `A`, `'\u20B9'` is ₹ |

```
System.out.println("Line1\nLine2");        // two lines
System.out.println("Name:\tKanhaiya");     // tab spacing
System.out.println("Path: C:\\Users");     // Path: C:\Users
System.out.println("She said \"Hello\""); // She said "Hello"
char c = '\u0041';                         // 'A'
```

**Things to remember**

- Inside a single-quoted `char`, you only need to escape `'` (and `\`). Inside a string, you only need to escape `"` (and `\`). Escaping the other one is allowed but unnecessary.
- An invalid escape such as `"\q"` is a compile error.

- `\uXXXX` is special: the compiler translates it **before** it parses the file, so it works anywhere in source code, including comments. A `\u000a` (newline) inside a `//` comment ends the comment and the rest of the line runs as code. Avoid it.
- In text blocks (Java 15+) you can end a line with `\` to join it with the next line (line continuation).
- `println("a\nb")` uses `\n`, while `System.lineSeparator()` or `%n` in `printf` gives the correct separator for the current platform (`\r\n` on Windows).

## 2.3 Identifiers, Keywords and Reserved Words

### Identifiers

An **identifier** is a name you give to a class, variable, method, package, interface or other element.

**Rules (enforced by the compiler)**

1. It may contain letters (including Unicode letters), digits, the underscore `_` and the dollar sign `$`.
2. It **cannot start with a digit**. `2nd` is invalid; `second2` is valid.
3. It cannot contain spaces or other symbols such as `-`, `@`, `#`, `%`. (`my-var` is read as `my minus var`.)
4. It is case-sensitive and has no length limit.
5. It cannot be a keyword or one of the literals `true`, `false`, `null`.
6. A lone `_` is not usable as an ordinary name (it is reserved for unnamed variables and patterns in modern Java).

| Valid | Invalid | Why invalid |
|---|---|---|
| `total`, `_count`, `$price` | `1stPlace` | Starts with a digit |
| `maxValue2`, `MAX_SIZE` | `max-value` | Hyphen not allowed |
| `naïve`, `सारांश` | `my var` | Space not allowed |
| `Class1`, `classes` | `class`, `true`, `null` | Keyword or literal |

**Naming conventions (not enforced, but every Java team follows them)**

| Element | Convention | Example |
|---|---|---|
| Class, interface, enum, record | PascalCase (UpperCamelCase), a noun | `OrderService`, `Runnable` |
| Method | camelCase, starts with a verb | `calculateTotal()`, `isEmpty()` |
| Variable, parameter | camelCase, meaningful | `customerName`, `retryCount` |
| Constant (`static final`) | UPPER_SNAKE_CASE | `MAX_RETRIES`, `DEFAULT_TIMEOUT_MS` |
| Package | all lowercase, reverse domain | `com.example.payments` |
| Type parameter | single uppercase letter | `T`, `E`, `K`, `V` |

## Keywords

A **keyword** is a word with a fixed meaning to the compiler. You cannot use it as an identifier. Java has **50 reserved keywords**: 48 in active use, plus `const` and `goto`, which are reserved but have no function.

| Group | Keywords |
|---|---|
| Primitive types | `byte` `short` `int` `long` `float` `double` `char` `boolean` |
| Control flow | `if` `else` `switch` `case` `default` `for` `while` `do` `break` `continue` `return` |
| Exceptions | `try` `catch` `finally` `throw` `throws` `assert` |
| Class and object | `class` `interface` `enum` `extends` `implements` `new` `this` `super` `instanceof` `abstract` `final` `static` `void` `native` |
| Access and packaging | `public` `private` `protected` `package` `import` |
| Concurrency and memory | `synchronized` `volatile` `transient` |
| Floating-point | `strictfp` (no effect since Java 17; all floating-point is strict) |

| Group | Keywords |
| --- | --- |
| Reserved, unused | `const` `goto` |

**Why are `goto` and `const` reserved?** The designers decided Java would not have `goto` (it leads to tangled code; labeled `break` and `continue` cover the legitimate uses) and `const` (replaced by `final`). They reserved the words so that no one could use them as names and the compiler can give a clear error.

## Literals that look like keywords

`true`, `false` and `null` are **literals**, not keywords, but they are also reserved: you cannot use them as identifiers.

## Contextual (restricted) keywords

As Java evolved, new words were added without breaking old code that might already use them as names. These words are keywords only in specific contexts.

| Word | Meaning | Since |
| --- | --- | --- |
| `var` | Local variable type inference. A reserved type name: you can still name a variable `var`, but not a class | 10 |
| `yield` | Returns a value from a switch expression block | 14 |
| `record` | Declares a record | 16 |
| `sealed`, `permits`, `non-sealed` | Restrict which classes may extend a type | 17 |
| `when` | Guard in pattern-matching switch | 21 |
| `module`, `requires`, `exports`, `opens`, `uses`, `provides`, `to`, `with`, `open`, `transitive` | Only meaningful inside `module-info.java` | 9 |

```
int record = 5;      // legal! 'record' is only special where a type
declaration is expected
var list = new ArrayList<String>();   // var used as a type
// class var {}      // illegal: 'var' cannot be a type name
```

**Interview point:** "Reserved words" is the umbrella term: keywords, plus the three literals. "Keywords" is the 50-word list. "Contextual keywords" are not reserved at all.

## 2.4 Primitive Data Types

Java has two families of types: **primitive types** (raw values stored directly, no methods) and **reference types** (objects, arrays, strings, accessed through a reference). There are exactly **8 primitives**. Their sizes are fixed by the language, so an `int` is 32 bits on every machine.

| Type | Size | Range | Default (fields) | Typical use |
|---|---|---|---|---|
| `byte` | 8 bits | -128 to 127 | `0` | Raw binary data, file and network buffers |
| `short` | 16 bits | -32,768 to 32,767 | `0` | Rare; legacy formats, memory-tight arrays |
| `int` | 32 bits | -2,147,483,648 to 2,147,483,647 (about ±2.1 billion) | `0` | Default whole number: counters, indexes |
| `long` | 64 bits | -2^63 to 2^63-1 (about ±9.2 × 10^18) | `0L` | Timestamps (epoch millis), IDs, big counts |
| `float` | 32 bits | about ±3.4 × 10^38, 6 to 7 digits precision | `0.0f` | Graphics, large numeric arrays where memory matters |
| `double` | 64 bits | about ±1.8 × 10^308, 15 to 16 digits precision | `0.0d` | Default decimal number: science, statistics |

| Type | Size | Range | Default (fields) | Typical use |
|---|---|---|---|---|
| `char` | 16 bits, **unsigned** | 0 to 65,535 (`'\u0000'` to `'\uffff'`) | `'\u0000'` | A single UTF-16 code unit |
| `boolean` | not precisely defined (JVM uses 1 byte or a 4-byte int slot) | `true` or `false` only | `false` | Conditions and flags |

Things worth remembering:

- Whole numbers are **signed two's complement**. The range is asymmetric: `Byte.MIN_VALUE` is -128 but `MAX_VALUE` is 127, so `Math.abs(Integer.MIN_VALUE)` is still negative.
- `char` is the only unsigned type, and it is a number: `'A' + 1` is `66` (an `int`).
- `boolean` is not a number. `if (1)` and `boolean b = 0;` are compile errors, unlike C.
- Local variables have **no default**: using one before assigning it is a compile error. Defaults apply to fields and array elements only.
- Each primitive has constants: `Integer.MAX_VALUE`, `Long.MIN_VALUE`, `Double.MAX_VALUE`, `Byte.SIZE`.

## Overflow in whole-number types

Arithmetic on `int` and `long` **wraps around silently**; there is no exception.

```
int max = Integer.MAX_VALUE;     // 2147483647
System.out.println(max + 1);     // -2147483648  (wrapped)
System.out.println(Math.addExact(max, 1));   // throws ArithmeticException:
integer overflow

System.out.println(5 / 2);       // 2   (integer division truncates toward
zero)
System.out.println(-5 / 2);      // -2
System.out.println(-5 % 2);      // -1  (remainder takes the sign of the
dividend)
System.out.println(5 / 0);       // ArithmeticException: / by zero
```

Use `Math.addExact`, `multiplyExact`, `toIntExact` when overflow would be a bug (money, quantities). Use `long` for anything that can pass 2 billion, such as epoch milliseconds, which is why `System.currentTimeMillis()` returns `long`.

## Deep dive: float and double

**How they are stored (IEEE 754).** Both types use the IEEE 754 binary floating-point standard, the same format as in C, Python and JavaScript. A number is stored in scientific notation, but in base 2: **sign, exponent, mantissa (fraction)**.

```
value = (-1)^sign  x  1.mantissa (binary)  x  2^(exponent - bias)

float  (32 bits):  1 sign | 8 exponent | 23 mantissa    bias = 127
double (64 bits):  1 sign | 11 exponent | 52 mantissa   bias = 1023
```

| Property | `float` | `double` |
|---|---|---|
| Total bits | 32 | 64 |
| Mantissa bits (plus 1 implicit) | 23 (24 effective) | 52 (53 effective) |
| Decimal digits of precision | about 7 | about 15 to 16 |
| Max value | 3.4028235E38 | 1.7976931348623157E308 |
| Smallest positive (subnormal) | 1.4E-45 | 4.9E-324 |
| Literal | `3.14f` or `3.14F` (suffix required) | `3.14` (default) or `3.14d` |
| Wrapper class | `Float` | `Double` |

The exponent controls how large the number is (the "floating" point); the mantissa controls how many significant digits you get. Because the mantissa has a fixed number of bits, floating-point numbers have **limited precision, not limited size**.

**Why `0.1 + 0.2 != 0.3`.** In decimal, 1/3 is 0.3333... forever. In binary, the same happens with 0.1: it is `0.0001100110011...` repeating, so it cannot be stored exactly. Only fractions whose denominator is a power of 2 (0.5, 0.25, 0.125) are exact. Each value is stored as the nearest representable number, and the tiny errors add up.

```
System.out.println(0.1 + 0.2);            // 0.30000000000000004
System.out.println(0.1 + 0.2 == 0.3);     // false
System.out.println(0.1f + 0.2f);          // 0.3  (float's error is hidden by
its short precision; it is still inexact)
System.out.println(1.0 - 0.9);            // 0.09999999999999998
System.out.println(3 * 0.1);              // 0.30000000000000004
```

**Never compare floating-point results with `==`.** Compare with a small tolerance (epsilon):

```
double a = 0.1 + 0.2, b = 0.3;
boolean same = Math.abs(a - b) < 1e-9;     // true
```

**Special values.** Floating-point never throws on division by zero; it returns special values:

```
System.out.println(1.0 / 0);      // Infinity
System.out.println(-1.0 / 0);     // -Infinity
System.out.println(0.0 / 0.0);    // NaN  (Not a Number)
System.out.println(Math.sqrt(-1));// NaN
System.out.println(1 / 0);        // ArithmeticException (INTEGER division
throws!)

double nan = Double.NaN;
System.out.println(nan == nan);           // false: NaN is not equal to
anything, even itself
System.out.println(Double.isNaN(nan));    // true: the correct test
System.out.println(0.0 == -0.0);          // true (negative zero exists)
System.out.println(Double.valueOf(0.0).equals(-0.0));  // false (equals
compares bit patterns)
```

**Precision loss with large values.** Not every integer fits exactly in a float:

```
float f = 16_777_217;             // 2^24 + 1
System.out.println(f);            // 1.6777216E7  (the +1 is lost: float has
24 effective bits)

long big = 9_007_199_254_740_993L; // 2^53 + 1
double d = big;                   // widening conversion, but loses
precision
System.out.println((long) d);     // 9007199254740992
```

`int` to `float`, `long` to `float` and `long` to `double` are legal widening conversions but **can lose precision**.

**Literal rules.**

```
float  f1 = 3.14;       // COMPILE ERROR: 3.14 is a double; possible lossy
conversion
float  f2 = 3.14f;      // OK
double d1 = 3.14;       // OK
double d2 = 1e3;        // 1000.0  (scientific notation)
double d3 = 5;          // OK: int widens to double, d3 = 5.0
float  f3 = 10;         // OK: int widens to float
```

**Printing and rounding.**

```
System.out.println(Math.round(2.5));   // 3  (returns long; rounds half up)
System.out.println(Math.round(-2.5));  // -2 (half rounds toward positive
infinity)
System.out.println(Math.floor(-2.5));  // -3.0
System.out.println(Math.ceil(-2.5));   // -2.0
System.out.println((int) 3.99);        // 3 (cast truncates, does not round)
System.out.printf("%.2f%n", 3.14159);  // 3.14 (formatting for display)
```

**When to use which.**

| Use `double` when | Use `float` when | Use neither when |
|---|---|---|
| Default choice for decimals; science, statistics, geometry, machine learning code on the CPU | Memory is the constraint: huge arrays of values (graphics, GPU buffers, sensor data), or an API demands it | You need exact decimals (money, tax, billing): use `BigDecimal` or integer minor units |

On modern 64-bit CPUs `double` is usually just as fast as `float`, so prefer `double` unless memory is a concern.

**`strictfp`.** Before Java 17, floating-point results could vary slightly between platforms unless you marked code `strictfp`. Since Java 17 all floating-point arithmetic is strict by default and the keyword has no effect.

## BigDecimal: exact decimal arithmetic

`java.math.BigDecimal` stores a number as an unscaled integer and a scale, so decimals are exact. It is the standard answer for financial calculations.

```
import java.math.BigDecimal;
import java.math.RoundingMode;

BigDecimal a = new BigDecimal("0.1");      // use the String constructor
BigDecimal b = new BigDecimal("0.2");
System.out.println(a.add(b));              // 0.3   (exact)

System.out.println(new BigDecimal(0.1));   //
0.1000000000000000055511151231257827...  (the double's error!)
System.out.println(BigDecimal.valueOf(0.1)); // 0.1  (valueOf uses
Double.toString)

BigDecimal price = new BigDecimal("199.99");
BigDecimal total = price.multiply(new BigDecimal("3"))
                        .setScale(2, RoundingMode.HALF_UP);   // always give
a rounding mode for division
System.out.println(total);                 // 599.97

System.out.println(new BigDecimal("1.0").equals(new BigDecimal("1.00")));
// false (scale differs)
System.out.println(new BigDecimal("1.0").compareTo(new BigDecimal("1.00")));
// 0 (numerically equal)
```

Rules of thumb: construct from `String` (or use `valueOf`), never from a `double` literal; use `compareTo`, not `equals`, for numeric equality; `divide` without a scale or `MathContext` throws `ArithmeticException` for non-terminating results such as 1/3. A common alternative for money is storing **minor units in a `long`** (paise or cents).

## Type conversion between primitives

**1. Widening (implicit, automatic).** A smaller or less-precise type goes into a larger one. Safe in range, no cast needed.

```
byte  ->  short  ->  int  ->  long  ->  float  ->  double
              char  ->  int
```

```
int i = 100;
long l = i;         // widening
double d = l;       // widening
char c = 'A';
int code = c;       // 65
```

`boolean` does not convert to or from any other type. `byte` and `short` convert to `char` only with a cast, because `char` is unsigned.

**2. Narrowing (explicit, a cast is required).** Going to a smaller type may lose data, so you must say so with `(type)`.

```
double d = 9.99;
int i = (int) d;            // 9      fractional part is dropped (truncation,
not rounding)

int big = 130;
byte b = (byte) big;        // -126   keeps the low 8 bits: 130 - 256

long l = 3_000_000_000L;
int x = (int) l;            // -1294967296   high bits discarded

double nan = Double.NaN;
int n = (int) nan;          // 0
int inf = (int) 1e20;       // 2147483647 (saturates to Integer.MAX_VALUE)
```

Narrowing from floating-point to integer **truncates toward zero** and saturates at the target range. Narrowing between integer types keeps only the lowest bits.

**3. Assignment of constants.** If an `int` **constant expression** fits in the target, the compiler narrows it automatically:

```
byte b = 10;        // OK, 10 fits and is a constant
byte c = 200;       // ERROR: does not fit
char ch = 65;       // OK, 'A'
int v = 10;
byte d = v;         // ERROR: v is not a constant
final int w = 10;
byte e = w;         // OK: final variable with constant value is a constant
expression
```

**4. Binary numeric promotion in expressions.** Before any arithmetic, operands are promoted:

- If either operand is `double`, the other becomes `double`.
- Otherwise, if either is `float`, both become `float`.
- Otherwise, if either is `long`, both become `long`.

- Otherwise **both become `int`**. This applies to `byte`, `short` and `char` operands, so their results are `int`.

```
byte a = 10, b = 20;
// byte c = a + b;        // ERROR: a + b is int
byte c = (byte) (a + b);  // OK

short s = 5;
s += 3;                   // OK: compound assignment adds a hidden cast
// s = s + 3;             // ERROR

int x = 7;
double avg = x / 2;       // 3.0   integer division happens first!
double avg2 = x / 2.0;    // 3.5
double avg3 = (double) x / 2;   // 3.5 (cast applies to x before dividing)
long millis = 1000 * 60 * 60 * 24 * 365;  // overflow: computed in int, then
widened
long right = 1000L * 60 * 60 * 24 * 365;  // correct
```

**5. `char` arithmetic.**

```
char c = 'a';
c++;                          // 'b'  (++ and += keep the type)
int n = c + 1;                // 99
char next = (char) (c + 1);  // 'c'
System.out.println('a' + 'b');         // 195 (int addition!)
System.out.println("" + 'a' + 'b');    // ab
System.out.println((char) ('A' + 2));  // C
int digit = '7' - '0';                  // 7 (converting a digit character to
its value)
```

## Wrapper classes (a first look)

Each primitive has an object twin in `java.lang`: `Byte`, `Short`, `Integer`, `Long`, `Float`, `Double`, `Character`, `Boolean`. They are needed because **generics and collections work only with objects** (`List<Integer>`, never `List<int>`), because a primitive cannot be `null`, and because they provide utilities (`Integer.parseInt`, `Integer.compare`, `Character.isDigit`).

```
Integer a = Integer.valueOf(10);   // boxing, explicit
Integer b = 10;                    // autoboxing: compiler inserts
Integer.valueOf(10)
int c = b;                         // unboxing: compiler inserts b.intValue()

List<Integer> list = new ArrayList<>();
list.add(5);                       // autoboxed
int first = list.get(0);           // auto-unboxed

int n = Integer.parseInt("123");   // String -> int
String s = Integer.toString(45);   // int -> String
```

**The Integer cache trap.** `Integer.valueOf` caches objects for values **-128 to 127**, so autoboxed values in that range are the same object. Outside that range, each boxing creates a new object, and `==` compares references.

```
Integer x = 127, y = 127;
System.out.println(x == y);        // true  (same cached object)
Integer p = 128, q = 128;
System.out.println(p == q);        // false (two different objects)
System.out.println(p.equals(q));   // true  (always use equals for wrappers)
```

**Unboxing NullPointerException.** Unboxing a `null` wrapper fails at runtime:

```
Integer count = map.get("missing");   // null
int total = count + 1;                // NullPointerException
```

**Performance note.** Boxing allocates objects (outside the cache). A loop like `Long sum = 0L; for (...) sum += i;` creates millions of temporary `Long` objects; declare `long sum` instead.

The cache range, `Character` and `Boolean` caches, and `Integer.valueOf` internals come back in a later section on the String pool and object model.

## 2.5 Variables in Java

A **variable** is a named location in memory that holds a value of a specific type. Declaring one tells the compiler the type and the name; the type decides how much memory is used and which operations are allowed.

```
int age;              // declaration
age = 25;             // assignment
int count = 10;       // declaration + initialization
int a = 1, b = 2, c;  // several of the same type in one statement
double price = 99.5;
String name = "Asha"; // a reference variable: holds a reference to a String
object
```

For a primitive variable the memory slot holds the value itself. For a **reference variable** (class, array, interface type) the slot holds a reference (an address-like handle) to an object on the heap; `null` means it refers to nothing.

Java has three kinds of variables, distinguished by **where they are declared**.

## Local variables

Declared inside a method, constructor or block (including loop headers and method parameters).

- Created when execution enters the block, destroyed when it exits. They live in the method's stack frame (primitives directly; for objects only the reference is in the frame, the object is on the heap).
- **No default value.** The compiler enforces *definite assignment*: you must assign before you read.
- Visible only inside the block where declared (block scope). Not allowed to be `public`, `private` or `static`.
- A lambda or inner class may use a local variable only if it is **final or effectively final** (never reassigned).

```
void demo() {
    int total;
    // System.out.println(total);   // COMPILE ERROR: variable total might
not have been initialized
    total = 5;
    System.out.println(total);      // OK

    for (int i = 0; i < 3; i++) { }
    // System.out.println(i);       // ERROR: i is out of scope here
}
```

## Instance variables (fields)

Declared inside a class, outside any method, **without** `static`.

- Each object gets its **own copy**, created with `new` and living on the heap with the object until it is garbage collected.
- **Automatic default values**: `0` for numeric types, `'\u0000'` for `char`, `false` for `boolean`, `null` for references.
- Can have access modifiers (`private`, `public`, …) and `final`.

## Static variables (class variables)

Declared with `static` inside a class.

- **One copy per class**, shared by all objects. Created when the class is initialized (see Section 1, class loading), not per object. In HotSpot they are stored with the class's `Class` object on the heap.
- Same default values as instance variables.
- Accessed through the class name: `Counter.total`. (Accessing through an object compiles but is misleading style.)
- Mutable statics are shared state, so they are a common source of bugs in multithreaded code.

```
public class Counter {
    static int total = 0;      // static: one copy for the whole class
    int id;                    // instance: every Counter has its own
    String label;              // instance, default null

    Counter() {
        total++;               // shared counter
        this.id = total;       // per-object id
    }

    public static void main(String[] args) {
        Counter a = new Counter();
        Counter b = new Counter();
        System.out.println(a.id + " " + b.id);   // 1 2
        System.out.println(Counter.total);        // 2
        System.out.println(a.label);              // null (default)
    }
}
```

## Side by side

| | Local | Instance | Static |
|---|---|---|---|
| Declared | Inside method, constructor or block | In class, no `static` | In class, with `static` |
| Copies | One per method call | One per object | One per class |
| Memory | Stack frame | Heap (inside the object) | With the class (Metaspace metadata; value held with the `Class` object on the heap) |
| Default value | **None** (must assign) | Yes (0, false, null) | Yes (0, false, null) |
| Lifetime | Until the block ends | As long as the object lives | As long as the class is loaded |
| Access modifiers | Not allowed | Allowed | Allowed |
| Accessed via | Name, in scope only | Object reference | Class name |

## Variable shadowing and `this`

A local variable or parameter with the same name as a field **shadows** it inside that method. Use `this.` to reach the field.

```
class Person {
    String name;
    Person(String name) {      // parameter shadows the field
        this.name = name;      // this.name = field, name = parameter
        // name = name;        // a classic bug: assigns the parameter to
itself, field stays null
    }
}
```

Java does **not** allow a local variable to shadow another local variable or parameter in an enclosing scope of the same method (`int x = 1; { int x = 2; }` is a compile error), but a local may shadow a field.

### Other points

- **`var` (Java 10+)** lets the compiler infer a local variable's type: `var names = new ArrayList<String>();`. The type is still fixed at compile time. It works only for locals with an initializer.
- **`final` variables** can be assigned once. `final int x; x = 5;` (a *blank final*) is legal for a local or a field assigned in every constructor.
- **Initialization order** for a class: static fields and static blocks (once, top to bottom), then for each object: instance fields and instance blocks, then the constructor body.

## 2.6 Literals and Constants

A **literal** is a fixed value written directly in source code: `42`, `3.14`, `'A'`, `"hello"`, `true`, `null`.

### Integer literals

An integer literal is an `int` by default. Four number systems are allowed:

| System | Prefix | Digits | Example | Decimal value |
|---|---|---|---|---|
| Decimal | none | 0-9 | `255` | 255 |
| Octal | leading `0` | 0-7 | `0377` | 255 |
| Hexadecimal | `0x` or `0X` | 0-9, a-f (any case) | `0xFF` | 255 |
| Binary (Java 7+) | `0b` or `0B` | 0, 1 | `0b1111_1111` | 255 |

```
int dec = 100;
int oct = 0144;       // 100 in octal notation
int hex = 0x64;       // 100
int bin = 0b1100100;  // 100

int wrong = 09;       // COMPILE ERROR: 9 is not an octal digit
System.out.println(010 + 010);   // 16   (octal 10 is decimal 8) -- a
favourite trap
int minusOne = 0xFFFFFFFF;       // -1: hex and binary literals fill all 32
bits, so the sign bit can be set
long big = 3_000_000_000L;       // needs L: 3000000000 does not fit in int
// int tooBig = 2147483648;      // ERROR: integer number too large
int min = -2147483648;           // OK: the compiler treats the minus as part
of the literal
```

- A `long` literal needs the suffix `L` (use uppercase; lowercase `l` looks like `1`).
- There is **no suffix** for `byte` or `short`; an `int` constant that fits is assigned directly (`byte b = 10;`).
- Binary literals are handy for bit masks and flags: `int FLAGS = 0b0000_1010;`.

## Floating-point literals

```
double d1 = 3.14;        // double by default
double d2 = 3.14d;       // suffix optional
float  f1 = 3.14f;       // suffix required for float
double d3 = 1.5e3;       // 1500.0 (scientific notation)
double d4 = 2.5E-3;      // 0.0025
double d5 = .5;          // 0.5
double d6 = 5.;          // 5.0
double d7 = 0x1.0p3;     // 8.0 (hexadecimal floating-point; rare, exponent
'p' is mandatory)
```

## Character, String, boolean and null literals

```
char c1 = 'A';               // single quotes, exactly one UTF-16 code unit
char c2 = 65;                // also 'A' (numeric value)
char c3 = '\u0041';          // Unicode escape
char c4 = '\n';              // escape sequence

String s = "Hello";          // double quotes; an object, interned in the
String pool
String block = """
        Multi-line text block
          keeps its layout
        """;                  // Java 15+

boolean ok = true;           // only true or false (lowercase)
String nothing = null;       // null can be assigned only to reference types
// int n = null;             // ERROR
```

## Java 7 enhancements to literals

Java 7 added two things: **binary literals** ( `0b...` , shown above) and **underscores in numeric literals**, to make long numbers readable. The compiler removes the underscores; they have no effect on the value.

```
int million = 1_000_000;
long cardNo = 1234_5678_9012_3456L;
int mask = 0b1010_1010;
int rgb = 0xFF_EC_DE;
double pi = 3.141_592_653;
int octal = 0_52;               // legal: octal 52
```

**Where underscores are NOT allowed** (each is a compile error):

| Invalid | Reason |
|---|---|
| `_100` , `100_` | Cannot start or end a number ( `_100` would be read as an identifier) |
| `3_.14` , `3._14` | Cannot be next to the decimal point |
| `100_L` , `3.14_f` | Cannot be next to a suffix `L` , `F` , `D` |
| `0x_FF` , `0b_101` | Cannot be right after the `0x` or `0b` prefix |

| Invalid | Reason |
| --- | --- |
| `0_x52` | Cannot split the prefix |

Underscores are literals-only: `Integer.parseInt("1_000")` throws `NumberFormatException`.

## Constants

Java has no `const` keyword in use (it is reserved but unused). A constant is created with **`final`**.

```
final double TAX_RATE = 0.18;         // local constant
final int MAX;                        // blank final: must be assigned
exactly once before use

public class Config {
    public static final int MAX_USERS = 100;          // class-level
constant: static final
    public static final String APP_NAME = "Billing";  // naming:
UPPER_SNAKE_CASE
    public static final double PI = 3.14159;
}
```

- **`final` variable:** can be assigned only once. A second assignment is a compile error.
- **`static final`:** one shared constant per class, named in UPPER_SNAKE_CASE, accessed as `Config.MAX_USERS`.
- **`final` reference** means the *reference* cannot change, not that the object is immutable.

```
final List<String> names = new ArrayList<>();
names.add("Asha");        // allowed: the object is mutable
// names = new ArrayList<>();   // ERROR: cannot reassign the reference
```

**Compile-time constants vs runtime constants.** A `final` variable of a primitive or `String` type that is initialized with a constant expression is a **compile-time constant**. The compiler inlines its value at every use.

```
static final int A = 5;                        // compile-time constant
static final int B = Integer.parseInt("5");    // runtime constant (value
known only when the class initializes)
static final int C = A * 2;                    // compile-time constant
(constant expression)
```

Why it matters: only compile-time constants can be used as `case` labels in a `switch`; reading one does **not** trigger class initialization; and because the value is copied into other classes, changing it requires **recompiling every class that uses it**, or they keep the old value. For constants that may change between releases, use a runtime constant (initialize through a method or `static` block).

For a fixed set of related named values (days, states, roles) prefer an **`enum`** over several `static final int` constants.

## 2.7 Operators in Java

An **operator** is a symbol that performs an operation on one, two or three **operands**. Operators are grouped by what they do.

### Arithmetic operators

| Operator | Meaning | Example | Result |
|---|---|---|---|
| `+` | Addition (also string concatenation) | `7 + 2` | 9 |
| `-` | Subtraction | `7 - 2` | 5 |
| `*` | Multiplication | `7 * 2` | 14 |
| `/` | Division | `7 / 2` and `7 / 2.0` | 3 and 3.5 |
| `%` | Remainder (modulus) | `7 % 2` | 1 |

```
System.out.println(7 / 2);       // 3     both operands are int: integer
division, fraction discarded
System.out.println(7 / 2.0);     // 3.5   one operand is double
System.out.println(-7 / 2);      // -3    truncates toward zero (not floor)
System.out.println(-7 % 2);      // -1    sign follows the left operand
System.out.println(7.5 % 2);     // 1.5   % works on floating-point too
System.out.println(Math.floorMod(-7, 2));   // 1   true mathematical modulus
System.out.println(10 / 0);      // ArithmeticException
System.out.println(10.0 / 0);    // Infinity
```

## Unary operators

| Operator | Meaning |
|---|---|
| `+x` , `-x` | Positive, negation |
| `++x` , `--x` | **Pre**-increment/decrement: change first, then use the value |
| `x++` , `x--` | **Post**-increment/decrement: use the value first, then change |
| `!b` | Logical NOT (boolean only) |
| `~n` | Bitwise complement (integers) |

```
int a = 5;
int b = a++;     // b = 5, a = 6   (use, then increment)
int c = ++a;     // a = 7, c = 7   (increment, then use)

int x = 5;
int y = x++ + ++x;   // 5 + 7 = 12; x ends at 7

int i = 3;
i = i++;         // i is STILL 3: the old value 3 is stored back after the
increment

byte by = 127;
by++;            // -128 (overflow wraps; ++ includes an implicit cast)
```

## Relational (comparison) operators

`==`, `!=`, `<`, `>`, `<=`, `>=` always produce a `boolean`.

- `==` on **primitives** compares values. On **references** it compares whether both refer to the *same object*. Use `equals()` for content (see Strings below).
- `<`, `>`, `<=`, `>=` work on numeric types and `char`, not on `boolean` or objects.
- Never chain: `1 < x < 10` is a compile error. Write `x > 1 && x < 10`.

## Logical operators

| Operator | Name | Behaviour |
|---|---|---|
| `&&` | Short-circuit AND | If the left side is `false`, the right side is **not evaluated** |
| `\|\|` | Short-circuit OR | If the left side is `true`, the right side is **not evaluated** |
| `!` | NOT | Inverts a boolean |
| `&` | Non-short-circuit AND | **Always** evaluates both sides |
| `\|` | Non-short-circuit OR | **Always** evaluates both sides |
| `^` | XOR | `true` when exactly one side is `true` |

```
String s = null;
if (s != null && s.length() > 3) { }   // safe: length() is never called when
s is null
if (s != null & s.length() > 3) { }    // NullPointerException: & evaluates
both sides

int n = 0;
boolean r1 = (n > 0) && (++n > 0);   // false, n stays 0 (right side skipped)
boolean r2 = (n > 0) & (++n > 0);    // false, n becomes 1 (right side ran)
```

Use `&&` and `||` by default. Use `&` and `|` on booleans only when the right side must run for its side effect, which is rare and usually a design smell.

## Bitwise and shift operators

These work on the **binary representation** of integer types ( `byte` , `short` , `char` , `int` , `long` ), after promotion to `int` or `long` .

| Operator | Name | Example (using 5 = 0101, 3 = 0011) | Result |
|---|---|---|---|
| `&` | AND | `5 & 3` = 0001 | 1 |
| `\|` | OR | `5 \| 3` = 0111 | 7 |
| `^` | XOR | `5 ^ 3` = 0110 | 6 |
| `~` | Complement (flip all bits) | `~5` | -6 (because `~n == -n - 1` ) |
| `<<` | Left shift: fills with 0 | `1 << 3` | 8 (multiplies by 2^n) |
| `>>` | Signed right shift: keeps the sign bit | `-8 >> 1` | -4 (divides by 2^n, rounding down) |
| `>>>` | Unsigned right shift: fills with 0 | `-8 >>> 1` | 2147483644 |

```
System.out.println(Integer.toBinaryString(5));    // 101
System.out.println(Integer.toBinaryString(-8));   //
11111111111111111111111111111000
System.out.println(-8 >>> 28);                    // 15
System.out.println(1 << 33);   // 2 : the shift distance is masked to 5 bits
for int (33 & 31 = 1)
System.out.println(1L << 33);  // 8589934592 : use a long for a 64-bit shift
```

### Useful idioms

```
boolean isOdd = (n & 1) == 1;                 // check parity
boolean powerOfTwo = n > 0 && (n & (n - 1)) == 0;
int withBitSet = flags | (1 << 3);            // set bit 3
int withBitCleared = flags & ~(1 << 3);       // clear bit 3
boolean hasBit = (flags & (1 << 3)) != 0;     // test bit 3
a ^= b; b ^= a; a ^= b;                       // swap two ints without a temp
(a curiosity, not good style)
```

## Assignment operators

`=` assigns. Compound forms combine an operation and an assignment: `+=`, `-=`, `*=`, `/=`, `%=`, `&=`, `|=`, `^=`, `<<=`, `>>=`, `>>>=`.

A compound assignment **includes an implicit cast** to the left-hand type, so it compiles where the long form would not:

```
byte b = 10;
// b = b + 5;       // ERROR: b + 5 is int
b += 5;              // OK: b = (byte)(b + 5)

int x = 10;
x *= 2.5;            // OK: x = (int)(x * 2.5) = 25   (silent narrowing!)

int p, q, r;
p = q = r = 7;       // assignment is an expression; evaluates right to left
```

## Ternary (conditional) operator

`condition ? valueIfTrue : valueIfFalse` is the only operator with three operands, and it is an **expression** (it produces a value).

```
int max = (a > b) ? a : b;
String label = (score >= 40) ? "Pass" : "Fail";
String size = n < 10 ? "small" : n < 100 ? "medium" : "large";   // nesting
is legal; keep it short
```

Trap: mixing a primitive and `null` unboxes: `int v = flag ? 1 : null;` throws `NullPointerException` when `flag` is false. Also, `true ? 1 : 2.0` yields `1.0`, because the branches are promoted to a common type (`double`).

## `instanceof`

Tests whether an object is an instance of a type (or a subtype). It is `false` for `null`.

```
Object o = "hello";
if (o instanceof String) { String s = (String) o; }   // classic: test, then
cast
if (o instanceof String s && s.length() > 3) {        // Java 16+ pattern
matching: test, cast and bind in one step
    System.out.println(s.toUpperCase());
}
System.out.println(null instanceof Object);           // false
```

## Other operators

`.` (member access), `[]` (array index), `()` (call, grouping), `new` (create object), `(type)` (cast), `->` (lambda and switch arrow), `::` (method reference).

## String concatenation with `+`

If either operand of `+` is a `String`, the other is converted to a string and the two are joined. Because `+` evaluates **left to right**, the position of the string matters.

```
System.out.println(1 + 2 + "3");     // 33     (1 + 2 = 3 first, then "3"
appended)
System.out.println("1" + 2 + 3);     // 123    (string from the start: "12"
then "123")
System.out.println("1" + (2 + 3));   // 15
System.out.println('a' + 'b' + "c"); // 195c   (char + char is int addition)
System.out.println("c" + 'a' + 'b'); // cab
System.out.println("x" + null);      // xnull
System.out.println("" + 1.0f);       // 1.0

String s = "a";
s += 1 + 2;                          // a3  (right side evaluated first)
```

## Operator precedence and associativity

Higher rows bind tighter. When in doubt, add parentheses.

| Level | Operators | Associativity |
|---|---|---|
| 1 | `expr++` `expr--` `.` `[]` `()` | Left to right |
| 2 | `++expr` `--expr` `+expr` `-expr` `~` `!` | Right to left |

| Level | Operators | Associativity |
|---|---|---|
| 3 | `new` and cast `(type)` | Right to left |
| 4 | `*` `/` `%` | Left to right |
| 5 | `+` `-` | Left to right |
| 6 | `<<` `>>` `>>>` | Left to right |
| 7 | `<` `<=` `>` `>=` `instanceof` | Left to right |
| 8 | `==` `!=` | Left to right |
| 9 | `&` | Left to right |
| 10 | `^` | Left to right |
| 11 | `\|` | Left to right |
| 12 | `&&` | Left to right |
| 13 | `\|\|` | Left to right |
| 14 | `? :` | Right to left |
| 15 | `=` `+=` `-=` ... `->` | Right to left |

### Common traps

```
int r = 2 + 3 * 4;            // 14, not 20
boolean t = a & b == c;       // parsed as a & (b == c): == binds tighter
than &
int s = 1 << 2 + 1;           // 8: + binds tighter than <<, so 1 << 3
boolean u = x > 0 || y > 0 && z > 0;   // && before ||: x>0 || (y>0 && z>0)
int v = a + b * c++;          // operands evaluate left to right; precedence
only decides grouping
```

Precedence decides **grouping**, not **evaluation order**. In Java operands are always evaluated left to right ( `f() + g() * h()` calls `f`, then `g`, then `h`).

# 2.8 Control Structures in Java

By default, statements run top to bottom. **Control structures** change that order. Java has three kinds: **decision (selection)** statements choose a path, **loops** repeat a path, and **transfer (jump)** statements leave or skip part of a path.

## Decision statements

### `if`, `if-else`, `else-if` ladder

```
int marks = 72;

if (marks >= 40) {
    System.out.println("Pass");
}

if (marks >= 40) {
    System.out.println("Pass");
} else {
    System.out.println("Fail");
}

if (marks >= 75)      System.out.println("Distinction");
else if (marks >= 60) System.out.println("First class");   // this one runs
else if (marks >= 40) System.out.println("Pass");
else                  System.out.println("Fail");
```

Rules to remember:

- The condition must be a `boolean`. `if (x = 5)` is a compile error for an `int` (it works in C, not in Java). With a `boolean`, `if (flag = true)` compiles and silently assigns, so write `if (flag)`.
- Without braces only the **next single statement** belongs to the `if`. Always use braces; skipping them causes real bugs.
- In an `else-if` ladder, conditions are checked top to bottom and **only the first match runs**. Order from most specific to least.
- **Dangling else:** an `else` attaches to the nearest unmatched `if`, regardless of indentation.

```
if (a > 0)
    if (b > 0) System.out.println("both");
    else System.out.println("?");   // belongs to the INNER if (b > 0), not
(a > 0)
```

**Nested `if`:** an `if` inside another. Prefer combining conditions with `&&` or returning early to avoid deep nesting.

```
if (user != null && user.isActive() && user.getAge() >= 18) { /* ... */ }
```

### `switch` statement (classic)

Chooses among many constant values of one expression.

```
int day = 3;
switch (day) {
    case 1:
        System.out.println("Mon");
        break;
    case 2:
        System.out.println("Tue");
        break;
    case 3:                       // matches
        System.out.println("Wed");
        // no break: FALL-THROUGH into the next case
    case 4:
        System.out.println("Thu");   // also runs
        break;
    default:
        System.out.println("Other");
}
// prints: Wed, Thu
```

Rules:

- Allowed selector types: `byte`, `short`, `char`, `int` (and their wrappers `Byte`, `Short`, `Character`, `Integer`), `String` (Java 7+), and `enum`. Not allowed: `long`, `float`, `double`, `boolean`.
- Case labels must be **compile-time constants** (literals, or `final` variables initialized with constants) and **unique**.
- **Fall-through:** execution continues into the next case until a `break` (or `return` / `throw`) is hit. It is occasionally intentional (grouping cases) but usually a bug.

- `default` is optional, can appear anywhere (it runs only when nothing matches), and normally goes last.
- A `null` selector throws `NullPointerException` (except with `case null` in Java 21 pattern switches). String matching is case-sensitive and uses `equals`.

```
switch (month) {          // grouping with intentional fall-through
    case 12: case 1: case 2: season = "Winter"; break;
    case 3: case 4: case 5:  season = "Spring"; break;
    default:                 season = "Other";
}
```

### `switch` expression (Java 14+)

The modern form uses `->`, has **no fall-through**, allows several labels per case, and can **return a value**.

```
String name = switch (day) {
    case 1, 7       -> "Weekend";
    case 2, 3, 4, 5, 6 -> "Weekday";
    default         -> "Invalid";
};

int len = switch (text) {
    case "short" -> 1;
    case "long"  -> {
        System.out.println("computing");
        yield 10;                   // yield returns a value from a block
    }
    default -> 0;
};

switch (day) {                      // arrow form also works as a statement
    case 1 -> System.out.println("Mon");
    case 2 -> System.out.println("Tue");
    default -> System.out.println("Other");
}
```

A switch **expression must be exhaustive**: a `default` is required unless the type is an `enum` or sealed type and every constant or subtype is covered. In Java 21+ the selector may also be an object with **type patterns** (`case Integer i -> ...`), covered later.

## Loop statements

**`for` loop.** Best when the number of iterations is known.

```
// for (initialization; condition; update) { body }
for (int i = 1; i <= 5; i++) {
    System.out.print(i + " ");        // 1 2 3 4 5
}

for (int i = 0, j = 10; i < j; i++, j--) { }   // several variables
for (;;) { /* infinite: no init, condition or update */ }
```

Order of execution: initialization **once**, then repeat { check condition, run body, run update }. The loop variable declared in the header is visible only inside the loop.

**`while` loop (entry-controlled).** Checks the condition **before** every iteration; may run zero times.

```
int n = 12345, digits = 0;
while (n > 0) {
    n /= 10;
    digits++;
}
System.out.println(digits);    // 5
```

**`do-while` loop (exit-controlled).** Runs the body **first**, then checks. It runs **at least once**. Note the semicolon after the condition.

```
int choice;
do {
    choice = readMenuChoice();
} while (choice != 0);
```

**Enhanced `for` (for-each).** Iterates over an array or any `Iterable`, with no index.

```
int[] nums = {3, 5, 7};
for (int x : nums) {
    System.out.println(x);
}
for (String s : List.of("a", "b")) { }
```

Limitations: no access to the index, cannot go backwards, and assigning to the loop variable (`x = 0`) does not change the array. Structurally modifying a collection while iterating it with for-each throws `ConcurrentModificationException`.

| Loop | Checks condition | Minimum runs | Use when |
|---|---|---|---|
| `for` | Before each iteration | 0 | Known count or index needed |
| `while` | Before each iteration | 0 | Repeat until a condition changes |
| `do-while` | After each iteration | 1 | Must run once (menus, input validation) |
| for-each | Handled by iterator | 0 | Visit every element, index not needed |

**Nested loops** put one loop inside another. The inner loop completes fully for each pass of the outer one.

```
for (int i = 1; i <= 4; i++) {          // rows
    for (int j = 1; j <= i; j++) {      // columns
        System.out.print("* ");
    }
    System.out.println();
}
// *
// * *
// * * *
// * * * *
```

**Infinite loops** are written deliberately in servers and event loops (`while (true)` with a `break` or `return` inside). An accidental one usually means the condition never changes.

## Transfer (jump) statements

**`break`** exits the nearest enclosing loop or `switch` immediately.

```
for (int i = 1; i <= 10; i++) {
    if (i == 4) break;
    System.out.print(i + " ");     // 1 2 3
}
```

**`continue`** skips the rest of the current iteration and goes to the next one (in a `for`, the update still runs).

```
for (int i = 1; i <= 5; i++) {
    if (i % 2 == 0) continue;
    System.out.print(i + " ");     // 1 3 5
}
```

**Labeled `break` and `continue`** target an outer loop by name. They are the only structured way to leave several loops at once.

```
outer:
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        if (i * j == 4) break outer;        // leaves BOTH loops
        if (j == 2) continue outer;         // jumps to the next i
        System.out.println(i + "," + j);
    }
}
```

**`return`** ends the current method. In a `void` method it is bare (`return;`); otherwise it provides the result (`return total;`). It also leaves any loop it is in.

**`goto`** is a reserved word but **cannot be used**. Java's designers removed it because unrestricted jumps produce hard-to-follow code; labeled `break`, `continue`, exceptions and `return` cover the legitimate cases.

**Unreachable code** is a compile error: statements right after `break`, `continue`, `return` or `throw` in the same block, or a loop whose condition is the constant `false`.

```
while (true) { }
System.out.println("x");      // ERROR: unreachable statement
```

### Worked examples

```
// 1. Factorial
long fact = 1;
for (int i = 2; i <= n; i++) fact *= i;          // n <= 20 fits in a long

// 2. Prime check
boolean prime = n > 1;
for (int i = 2; (long) i * i <= n; i++) {
    if (n % i == 0) { prime = false; break; }
}

// 3. Fibonacci: first 10 terms
int a = 0, b = 1;
for (int i = 0; i < 10; i++) {
    System.out.print(a + " ");
    int next = a + b; a = b; b = next;
}

// 4. Reverse a number
int rev = 0;
for (int t = n; t != 0; t /= 10) rev = rev * 10 + t % 10;

// 5. FizzBuzz
for (int i = 1; i <= 15; i++) {
    if (i % 15 == 0)      System.out.println("FizzBuzz");
    else if (i % 3 == 0)  System.out.println("Fizz");
    else if (i % 5 == 0)  System.out.println("Buzz");
    else                  System.out.println(i);
}
```

## 2.9 Arrays and Strings

### Arrays

An **array** is a fixed-size, ordered collection of elements of the **same type**, stored contiguously and accessed by a zero-based index. In Java an array is an **object**: it lives on the heap, has a `length` field, and the variable holds a reference to it.

**Declaration, creation, initialization**

```
int[] marks;                        // declaration (preferred style: brackets
with the type)
int scores[];                       // legal C-style, avoid
marks = new int[5];                 // creation: 5 elements, all default 0
int[] ages = new int[]{18, 21, 30};
int[] primes = {2, 3, 5, 7};        // shorthand, only in a declaration
String[] names = new String[3];     // elements default to null
char[] letters = {'a', 'b', 'c'};
boolean[] flags = new boolean[4];   // all false

printArray(new int[]{1, 2, 3});     // anonymous array passed straight to a
method
```

Default element values are the same as field defaults: `0`, `0.0`, `'\u0000'`, `false`, `null`. The size is fixed once created; `length` is a **field** (`arr.length`), not a method (compare `String.length()` and `List.size()`).

### Access and traversal

```
int[] a = {10, 20, 30, 40};
System.out.println(a[0]);           // 10
a[2] = 99;
System.out.println(a.length);       // 4

for (int i = 0; i < a.length; i++) System.out.print(a[i] + " ");
for (int v : a) System.out.print(v + " ");

int last = a[a.length - 1];         // last element
// a[4]  -> ArrayIndexOutOfBoundsException: Index 4 out of bounds for length
4
// new int[-1] -> NegativeArraySizeException
```

Valid indexes are `0` to `length - 1`. Java checks bounds on **every access** at runtime; there are no silent buffer overflows as in C.

**Multi-dimensional and jagged arrays.** A 2-D array is an *array of arrays*, so rows can have different lengths.

```
int[][] grid = new int[3][4];             // 3 rows, 4 columns, all 0
int[][] m = {{1, 2, 3}, {4, 5, 6}};
System.out.println(m[1][2]);              // 6
System.out.println(m.length + " x " + m[0].length);   // 2 x 3

int[][] jagged = new int[3][];            // only the outer array is created
jagged[0] = new int[1];
jagged[1] = new int[3];
jagged[2] = new int[2];                   // rows of different sizes

for (int[] row : m) {
    for (int v : row) System.out.print(v + " ");
    System.out.println();
}
```

### Arrays are reference types: copying and comparing

```
int[] a = {1, 2, 3};
int[] b = a;              // NOT a copy: both variables point to the same
array
b[0] = 99;
System.out.println(a[0]); // 99

int[] c = a.clone();      // new array, same contents (shallow copy)
int[] d = Arrays.copyOf(a, 5);          // [99, 2, 3, 0, 0] (padded with
defaults)
int[] e = Arrays.copyOfRange(a, 1, 3);  // [2, 3]
System.arraycopy(a, 0, d, 1, 3);        // src, srcPos, dest, destPos, length
(fastest, native)

System.out.println(a == c);                 // false: different objects
System.out.println(Arrays.equals(a, c));    // true: same contents
System.out.println(a.equals(c));            // false: array equals() is
reference equality
System.out.println(a);                      // [I@1b6d3586 (type and hash,
not the contents)
System.out.println(Arrays.toString(a));     // [99, 2, 3]
System.out.println(Arrays.deepToString(m)); // [[1, 2, 3], [4, 5, 6]]
```

`clone()` and `copyOf` on a 2-D or object array copy only the **references**, so inner arrays and objects are shared (shallow). A deep copy needs a loop or copying each row.

### The `Arrays` utility class (`java.util.Arrays`)

| Method | Purpose |
| --- | --- |
| `Arrays.toString(a)`, `deepToString(a)` | Readable text of contents |
| `Arrays.sort(a)`, `sort(a, from, to)` | Sort in place. Primitives: dual-pivot quicksort. Objects: stable TimSort |
| `Arrays.sort(objs, comparator)` | Custom order (objects only, not `int[]`) |
| `Arrays.binarySearch(a, key)` | Binary search on a **sorted** array. Returns index, or `-(insertionPoint) - 1` if absent |
| `Arrays.fill(a, value)` | Set every element |
| `Arrays.copyOf`, `copyOfRange` | Copy with new length |
| `Arrays.equals`, `deepEquals` | Compare contents |
| `Arrays.asList(a)` | Fixed-size `List` view backed by the array (works on object arrays, not `int[]`) |
| `Arrays.stream(a)` | Stream of elements |
| `Arrays.hashCode(a)` | Content-based hash |

```
int[] nums = {5, 2, 9, 1};
Arrays.sort(nums);                                  // [1, 2, 5, 9]
System.out.println(Arrays.binarySearch(nums, 5)); // 2
System.out.println(Arrays.binarySearch(nums, 3)); // -3  (would insert at
index 2: -(2) - 1)
Integer[] boxed = {5, 2, 9, 1};
Arrays.sort(boxed, Collections.reverseOrder());  // [9, 5, 2, 1]
int sum = Arrays.stream(nums).sum();              // 17
```

### Typical array programs

```
// Maximum
int max = a[0];
for (int i = 1; i < a.length; i++) if (a[i] > max) max = a[i];

// Reverse in place
for (int i = 0, j = a.length - 1; i < j; i++, j--) {
    int t = a[i]; a[i] = a[j]; a[j] = t;
}

// Linear search
int idx = -1;
for (int i = 0; i < a.length; i++) if (a[i] == key) { idx = i; break; }

// Matrix transpose (square n x n)
for (int i = 0; i < n; i++)
    for (int j = i + 1; j < n; j++) { int t = m[i][j]; m[i][j] = m[j][i];
m[j][i] = t; }
```

**Object arrays and covariance.** `Object[] objs = new String[2];` compiles (arrays are covariant), but `objs[0] = 5;` then throws `ArrayStoreException` at runtime. Generics were designed to avoid this.

**Character arrays.** A `char[]` is a plain array of characters, mutable, and not a `String`.

```
char[] cs = {'J', 'a', 'v', 'a'};
System.out.println(cs);                 // Java   (println has a char[]
overload)
System.out.println("Lang: " + cs);      // Lang: [C@15db9742  (concatenation
gives the type and hash!)
String s = new String(cs);              // "Java"
String t = String.valueOf(cs);          // "Java"
char[] back = "Java".toCharArray();     // copy of the string's characters
Arrays.sort(back);                      // sorts characters: [J, a, a, v]
char[] password = new char[8];          // passwords are kept in char[] so
they can be wiped: Arrays.fill(password, '\0')
```

## Strings

`String` (`java.lang.String`) is a sequence of UTF-16 characters. It is an **object**, not a primitive, but it has special language support: literals in double quotes and the `+` operator.

**Immutability.** A `String` can never be changed after creation. Methods like `toUpperCase()` or `replace()` return a **new** string.

```
String s = "hello";
s.toUpperCase();               // result discarded: s is still "hello"
s = s.toUpperCase();           // s now refers to a new object "HELLO"
```

Why strings are immutable: **security** (class names, file paths, URLs and credentials cannot be altered after validation), **thread safety** (no locking needed), **hash caching** (the hash code is computed once, which makes strings excellent `HashMap` keys), and the **String pool** (sharing is safe only if nobody can change a shared object).

### The String pool and creation

The JVM keeps a pool of unique string objects (on the heap since Java 7). A literal is looked up in the pool and reused.

```
String a = "java";                 // pool
String b = "java";                 // same pooled object
String c = new String("java");     // new object on the heap (the literal is
also in the pool)
String d = c.intern();             // returns the pooled instance

System.out.println(a == b);        // true   same pooled object
System.out.println(a == c);        // false  different objects
System.out.println(a.equals(c));   // true   same characters
System.out.println(a == d);        // true

String x = "ja" + "va";            // compile-time constant, folded to "java"
System.out.println(a == x);        // true
String p = "ja";
String y = p + "va";               // computed at runtime: a new object
System.out.println(a == y);        // false
final String q = "ja";
System.out.println(a == q + "va"); // true: q is a constant variable
```

`new String("java")` can create **two** objects (the pooled literal if absent, plus the heap copy). It is almost never needed. **Always compare strings with `equals()`**, or `equalsIgnoreCase()`. Put the literal first to avoid a `NullPointerException`: `"yes".equals(input)`.

### Frequently used `String` methods

```
String s = "  Hello, Java World  ";
s.length();                         // 21   (UTF-16 code units, including
spaces)
s.charAt(2);                        // 'H'  (index out of range ->
StringIndexOutOfBoundsException)
s.isEmpty();                        // false (length 0)
s.isBlank();                        // false (empty or only whitespace) - Java
11
s.trim();                           // "Hello, Java World"  removes chars <=
U+0020
s.strip();                          // same, but Unicode-whitespace aware -
Java 11
s.stripLeading(); s.stripTrailing();

String t = "Hello, Java";
t.substring(7);                     // "Java"      from index 7 to the end
t.substring(0, 5);                  // "Hello"     begin inclusive, end
EXCLUSIVE
t.indexOf('a');                     // 8   first occurrence, -1 if none
t.indexOf("Java");                  // 7
t.lastIndexOf('a');                 // 10
t.contains("Jav");                  // true
t.startsWith("Hell");               // true
t.endsWith("va");                   // true
t.toUpperCase(); t.toLowerCase();
t.replace('a', 'o');                // "Hello, Jovo"   all occurrences,
literal
t.replace("Java", "World");         // "Hello, World"
t.replaceAll("[aeiou]", "*");       // regex-based: "H*ll*, J*v*"
t.concat("!");                      // "Hello, Java!"

"a,b,c".split(",");                 // ["a", "b", "c"]
"a,b,,".split(",");                 // ["a", "b"]    trailing empty strings
are dropped
"a,b,,".split(",", -1);             // ["a", "b", "", ""]
"a1b22c".split("\\d+");             // ["a", "b", "c"]   split takes a regex
String.join("-", "2026", "10", "31");        // "2026-10-31"
String.join(", ", List.of("x", "y"));        // "x, y"
"ab".repeat(3);                     // "ababab" - Java 11
"l1\nl2".lines().count();           // 2 - Java 11

"apple".compareTo("banana");        // -1   negative: before (difference of
first differing chars)
```

```
"b".compareTo("a");                 // 1
"apple".compareToIgnoreCase("APPLE");   // 0
"hello".equals("Hello");            // false
"hello".equalsIgnoreCase("Hello"); // true
```

### Concatenation and performance

Because strings are immutable, `s = s + x` in a loop creates a new string each time, copying all previous characters: total cost grows quadratically.

```
String s = "";
for (int i = 0; i < 100_000; i++) s += i;       // very slow: 100,000
intermediate strings

StringBuilder sb = new StringBuilder();
for (int i = 0; i < 100_000; i++) sb.append(i); // fast: one growing buffer
String result = sb.toString();
```

A single expression such as `"a" + x + "b"` is fine: the compiler turns it into an efficient single operation. The problem is only repeated concatenation **inside a loop**.

### `StringBuilder` vs `StringBuffer`

Both are **mutable** character sequences with the same methods. They differ in thread safety.

| | `String` | `StringBuilder` | `StringBuffer` |
|---|---|---|---|
| Mutable | No | Yes | Yes |
| Thread-safe | Yes (immutable) | **No** | Yes (methods `synchronized`) |
| Speed | Slow for repeated edits | **Fastest** | Slower (locking) |
| Since | 1.0 | Java 5 | 1.0 |
| Use | Fixed text, keys, constants | Default for building text in one thread | Legacy; rarely needed |

```
StringBuilder sb = new StringBuilder("Java");
sb.append(" 21").append('!');       // "Java 21!"   methods return this, so
calls chain
sb.insert(0, ">> ");                  // ">> Java 21!"
sb.reverse();                         // "!12 avaJ >>"
sb.reverse();
sb.deleteCharAt(sb.length() - 1);     // remove '!'
sb.delete(0, 3);                      // "Java 21"
sb.setCharAt(0, 'j');                 // "java 21"
sb.replace(0, 4, "JAVA");             // "JAVA 21"
int cap = sb.capacity();              // internal buffer size (default 16,
grows to (old * 2) + 2)
// StringBuilder has no equals(): sb1.equals(sb2) is reference comparison;
compare with toString().equals(...)
```

### Converting between strings and numbers

```
int n = Integer.parseInt("123");          // String -> int   ("12a" ->
NumberFormatException)
double d = Double.parseDouble("3.5");
long l = Long.parseLong("9000000000");
boolean b = Boolean.parseBoolean("TRUE"); // true; anything not "true"
(ignoring case) is false
Integer boxed = Integer.valueOf("77");

String s1 = String.valueOf(123);          // int -> String
String s2 = Integer.toString(123);
String s3 = "" + 123;                     // works, but less clear
String bin = Integer.toBinaryString(10);  // "1010"
String hex = Integer.toHexString(255);    // "ff"
int fromBin = Integer.parseInt("1010", 2); // 10
```

### Formatting with `String.format` and `printf`

```
System.out.printf("%s is %d years old%n", "Asha", 28);   // Asha is 28 years
old
System.out.printf("%.2f%n", 3.14159);                     // 3.14
System.out.printf("[%8.3f]%n", 3.14159);                  // [   3.142]
width 8, 3 decimals
System.out.printf("[%-8s]%n", "Java");                    // [Java    ]
left-aligned
System.out.printf("[%05d]%n", 42);                        // [00042]
System.out.printf("%,d%n", 1234567);                      // 1,234,567
System.out.printf("%x %c %b%n", 255, 'A', true);          // ff A true
String line = String.format("%-10s|%5d", "item", 7);
```

| Specifier | Meaning |
|---|---|
| `%d` | Integer ( `byte` , `short` , `int` , `long` ) |
| `%f` , `%.2f` | Floating-point, with decimal places |
| `%s` | String (calls `toString()` ) |
| `%c` | Character |
| `%b` | Boolean |
| `%x` , `%o` | Hexadecimal, octal |
| `%e` | Scientific notation |
| `%n` | Platform line separator (prefer over `\n` ) |
| `%%` | A literal percent sign |

`%d` with a `double` throws `IllegalFormatConversionException` . Decimal separators follow the default locale; pass a `Locale` to force one.

### Worked string programs

```
// Palindrome check
boolean pal = true;
for (int i = 0, j = s.length() - 1; i < j; i++, j--)
    if (s.charAt(i) != s.charAt(j)) { pal = false; break; }

// Count vowels
int vowels = 0;
for (char ch : s.toLowerCase().toCharArray())
    if ("aeiou".indexOf(ch) >= 0) vowels++;

// Anagram check
char[] x = s1.toCharArray(), y = s2.toCharArray();
Arrays.sort(x); Arrays.sort(y);
boolean anagram = Arrays.equals(x, y);

// Reverse words
String[] words = sentence.trim().split("\\s+");
Collections.reverse(Arrays.asList(words));
String reversed = String.join(" ", words);
```

## 2.10 Basic Input and Output

Programs need to show results and take data. In console programs this is done through three streams in the `System` class: `System.out` (standard output), `System.err` (standard error) and `System.in` (standard input).

### Output

```
System.out.println("Hello");         // prints text and a line break
System.out.print("No newline ");     // prints text only
System.out.println();               // just a blank line
System.out.println(42);             // overloaded for int, double, char,
boolean, char[], Object ...
System.out.println(3.0 + " and " + true);
System.out.printf("%s scored %d (%.1f%%)%n", "Asha", 45, 90.0);   //
formatted, no automatic newline
System.out.println(String.format("%05d", 42));    // 00042
System.err.println("This goes to the error stream");   // separate stream,
shown in red in many IDEs
```

`println` calls `String.valueOf(x)` on its argument, which for objects calls `toString()`. `printf` and `format` use the specifiers from section 2.9. Use `System.err` for diagnostics so they can be redirected separately from normal output (`java App > out.txt` redirects only `System.out`).

## Input with `Scanner`

`java.util.Scanner` reads and parses text from a source: the keyboard (`System.in`), a file or a string. By default it splits input on whitespace.

```
import java.util.Scanner;

public class InputDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter two numbers: ");
        int a = sc.nextInt();
        int b = sc.nextInt();
        System.out.println("Sum = " + (a + b));

        sc.close();
    }
}
```

| Method | Reads |
|---|---|
| `nextInt()`, `nextLong()`, `nextDouble()`, `nextBoolean()`, `nextByte()`, `nextShort()`, `nextFloat()` | The next token, converted to that type |
| `next()` | The next **token** (one word, stops at whitespace) |
| `nextLine()` | The **rest of the current line**, including spaces; the line break is consumed but not returned |
| `hasNext()`, `hasNextInt()`, `hasNextLine()` | Whether more input of that kind is available (use in loops) |

**There is no `nextChar()`.** Read a character with `sc.next().charAt(0)`.

### Scanner pitfalls

1. **The leftover newline.** `nextInt()`, `nextDouble()` and `next()` read only the token and leave the line break in the buffer. A following `nextLine()` then returns the empty remainder of the first line and appears to "skip" input.

```
System.out.print("Age: ");
int age = sc.nextInt();            // user types 28 and presses Enter
System.out.print("Name: ");
String name = sc.nextLine();       // returns "" immediately: it consumed the
leftover Enter

// Fix 1: consume the rest of the line first
int age2 = sc.nextInt();
sc.nextLine();                     // discard the newline
String name2 = sc.nextLine();

// Fix 2 (cleaner): read every value with nextLine() and convert yourself
int age3 = Integer.parseInt(sc.nextLine().trim());
String name3 = sc.nextLine();
```

2. **Wrong input type.** `nextInt()` on the text `abc` throws `InputMismatchException`. Check with `hasNextInt()` or catch the exception.
3. **No more input.** `nextInt()` or `nextLine()` at end of input throws `NoSuchElementException`. Loop with `while (sc.hasNextInt())`.
4. **Closing `System.in`.** `sc.close()` also closes `System.in`, so you cannot create another Scanner on it later. Create one Scanner for the whole program, or use try-with-resources in `main`.

```
try (Scanner sc = new Scanner(System.in)) {
    int n = sc.nextInt();
    int sum = 0;
    for (int i = 0; i < n; i++) sum += sc.nextInt();
    System.out.println("Total: " + sum);
}
```

**Faster input.** `Scanner` is convenient but slow for large input (it uses regular expressions). For big inputs, such as coding contests, use `BufferedReader`:

```
BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
int n = Integer.parseInt(br.readLine().trim());           // readLine()
returns null at end of input
StringTokenizer st = new StringTokenizer(br.readLine());  // split a line
into tokens
int x = Integer.parseInt(st.nextToken());
// main must declare:  throws IOException
```

**Other ways to get input**

- **Command-line arguments:** `java App 5 hello` puts `"5"` and `"hello"` in `args[0]` and `args[1]` (always strings; convert with `Integer.parseInt`).
- **`System.console()`:** returns `null` when the program is not attached to a terminal (typical in IDEs); useful for passwords via `readPassword()`.
- **Java 25:** recent JDKs add the simplified helpers `IO.println(...)` and `IO.readln(prompt)` in `java.lang.IO` for small programs.

## 2.11 Type Casting

**Casting** converts a value or reference from one type to another. It comes in two families.

### Primitive casting

Covered in detail in section 2.4. The rules in short:

| Direction | Syntax | Risk |
|---|---|---|
| **Widening** (small to large): `byte → short → int → long → float → double`, `char → int` | Implicit, automatic | None for range; `int`/`long` to `float`/`double` may lose precision |
| **Narrowing** (large to small) | Explicit: `(int) x` | Truncation, overflow, lost fractions |

```
int i = 'A';              // 65  (char widens to int)
char c = (char) 66;       // 'B' (int narrows to char, explicit)
char d = (char) (c + 1);  // 'C'
int code = (int) 'z';     // 122
double avg = (double) 7 / 2;      // 3.5  cast binds tighter than /
double avg2 = (double) (7 / 2);   // 3.0  cast applied after integer division
int r = (int) 3.99;       // 3
int rr = (int) Math.round(3.99);  // 4
long big = 5_000_000_000L;
int wrong = (int) big;    // 705032704   high bits lost
int exact = Math.toIntExact(big);   // throws ArithmeticException instead of
silently overflowing
```

`boolean` cannot be cast to or from any other primitive.

## Reference casting: upcasting and downcasting

For objects, a cast does **not** change the object. It only changes the **type of the reference** through which you see it. This works along an inheritance (or interface) hierarchy.

```
class Animal { void eat() { System.out.println("eating"); } }
class Dog extends Animal {
    @Override void eat() { System.out.println("dog eating"); }
    void bark() { System.out.println("woof"); }
}
class Cat extends Animal { }
```

**Upcasting** (subclass to superclass): always safe, so it is implicit.

```
Animal a = new Dog();     // upcast: a Dog is an Animal
a.eat();                  // "dog eating": the actual object's overridden
method runs (polymorphism)
// a.bark();              // COMPILE ERROR: type Animal has no bark(); the
reference type decides what you can call
```

**Downcasting** (superclass to subclass): may be unsafe, so you must write the cast, and it is **checked at runtime**.

```
Animal a = new Dog();
Dog d = (Dog) a;          // OK: the object really is a Dog
d.bark();

Animal b = new Cat();
Dog bad = (Dog) b;        // compiles, but throws ClassCastException at
runtime

// Safe pattern: test first
if (b instanceof Dog dog) {      // Java 16+ pattern matching
    dog.bark();
}
```

### Rules

- Casting between **unrelated** classes is a compile error: `String s = (String) Integer.valueOf(5);` will not compile. Casting up and down the same hierarchy compiles; the real type is checked when the program runs.
- `null` can be cast to any reference type without error; `null instanceof X` is `false`.
- Casting to an **interface** type is allowed for any non-final class, because a subclass might implement it; it can still fail with `ClassCastException`.
- Fields are not polymorphic: `((Animal) dog).name` reads `Animal`'s field if both classes declare one (field hiding). Methods are polymorphic; fields are not.

**Casting and boxing.** A cast between a primitive and its own wrapper works; across different types it does not.

```
Object o = 5;                 // autoboxed to Integer
int i = (int) o;              // OK: cast Object to Integer, then unbox
// long l = (long) o;         // ClassCastException: the object is an
Integer, not a Long
long ok = ((Integer) o).longValue();   // correct
// Long wrong = (Long) o;     // ClassCastException
// Integer boxed = 5; Long x = (Long) boxed;   // does not compile: unrelated
types
```

**Generics and casts.** Unchecked casts such as `(List<String>) obj` compile with a warning because the type argument is erased at runtime; the cast cannot verify the element type, so a later `ClassCastException` can appear far from the cast.

# Review Questions — Basic and Senior Level

Try every question before opening the answers. Part A is multiple choice (many are "predict the output" traps), Part B is short answer, Part C is coding practice, and Part D is senior level. All answers and explanations are consolidated in the next section.

## Part A — Multiple Choice

**Q1.** Which is a valid Java identifier?

- (a) `2total`
- (b) `total-2`
- (c) `_total$`
- (d) `class`

**Q2.** Which of these is NOT a keyword?

- (a) `goto`
- (b) `const`
- (c) `true`
- (d) `strictfp`

**Q3.** How many bits is a Java `char`?

- (a) 8
- (b) 16
- (c) 32
- (d) Depends on the platform

**Q4.** What happens here?

```
void f() { int x; System.out.println(x); }
```

- (a) Prints 0
- (b) Prints null
- (c) Compile error
- (d) Runtime exception

**Q5.** What is printed?

```
System.out.println(10 + 20 + "30" + 40 + 50);
```

- (a) `1020304050`
- (b) `30304050`
- (c) `3030`
- (d) `150`

**Q6.** What is `b` after this code?

```
byte b = 127;
b++;
```

- (a) 128
- (b) 127
- (c) -128
- (d) Compile error

**Q7.** What is `x`?

```
int x = 010 + 010;
```

- (a) 20
- (b) 16
- (c) 10
- (d) Compile error

**Q8.** What is printed?

```
System.out.println(0.1 + 0.2 == 0.3);
System.out.println(0.1f + 0.2f == 0.3f);
```

- (a) `true` then `true`
- (b) `false` then `false`
- (c) `false` then `true`
- (d) `true` then `false`

**Q9.** What is printed?

```
System.out.println(1 / 0.0);
```

- (a) `ArithmeticException`
- (b) `0.0`

- (c) `Infinity`
- (d) `NaN`

**Q10.** What is the result?

```
System.out.println(5 / 2 + 5 % 2 + 5 / 2.0);
```

- (a) 5.5
- (b) 6.0
- (c) 5
- (d) 5.0

**Q11.** Which line does NOT compile?

- (a) `float f = 3.14f;`
- (b) `float f = 3.14;`
- (c) `double d = 3;`
- (d) `long l = 100;`

**Q12.** Which statement compiles?

- (a) `short s = 5; s = s + 1;`
- (b) `short s = 5; s += 1;`
- (c) `byte b = 200;`
- (d) `int i = 3.5;`

**Q13.** What is printed?

```
Integer p = 128, q = 128;
Integer m = 100, n = 100;
System.out.println((p == q) + " " + (m == n));
```

- (a) `true true`
- (b) `false false`
- (c) `false true`
- (d) `true false`

**Q14.** What is printed?

```
String a = "hi";
String b = new String("hi");
String c = "h" + "i";
System.out.println((a == b) + " " + (a == c) + " " + a.equals(b));
```

- (a) `true true true`
- (b) `false true true`
- (c) `false false true`
- (d) `false true false`

**Q15.** How many `String` objects can `new String("abc")` create if `"abc"` is not yet in the pool?

- (a) 0
- (b) 1
- (c) 2
- (d) 3

**Q16.** Which type CANNOT be the selector of a classic `switch`?

- (a) `String`
- (b) `char`
- (c) `long`
- (d) an `enum`

**Q17.** What is `i`?

```
int i = 3;
i = i++;
```

- (a) 3
- (b) 4
- (c) 5
- (d) Compile error

**Q18.** What is printed?

```
System.out.println('a' + 'b');
System.out.println("" + 'a' + 'b');
```

- (a) `195` then `ab`
- (b) `ab` then `ab`
- (c) `195` then `195`
- (d) `ab` then `195`

**Q19.** What is `y`?

```
int x = 5;
int y = x++ + ++x;
```

- (a) 10
- (b) 11
- (c) 12
- (d) 13

**Q20.** What is printed?

```
System.out.println(-16 >> 2);
System.out.println(-16 >>> 28);
```

- (a) `-4` and `15`
- (b) `4` and `15`
- (c) `-4` and `-1`
- (d) `-64` and `0`

**Q21.** Which loop always executes its body at least once?

- (a) `for`
- (b) `while`
- (c) `do-while`
- (d) for-each

**Q22.** What is printed?

```
for (int i = 0; i < 5; i++) {
    if (i == 2) continue;
    if (i == 4) break;
    System.out.print(i);
}
```

- (a) `0123`

- (b) `013`
- (c) `01234`
- (d) `0134`

**Q23.** What does this do?

```
int[] a = new int[3];
System.out.println(a[3]);
```

- (a) Prints 0
- (b) Prints null
- (c) `ArrayIndexOutOfBoundsException`
- (d) Compile error

**Q24.** What is printed?

```
int[] a = {1, 2};
int[] b = a;
b[0] = 9;
System.out.println(a[0] + " " + a.equals(new int[]{9, 2}));
```

- (a) `1 false`
- (b) `9 true`
- (c) `9 false`
- (d) `1 true`

**Q25.** What is the length of the array?

```
System.out.println("a,b,,".split(",").length);
```

- (a) 2
- (b) 3
- (c) 4
- (d) 5

**Q26.** What is printed?

```
System.out.println(Math.round(-2.5));
System.out.println((int) -3.99);
```

- (a) `-3` and `-4`

- (b) `-2` and `-3`
- (c) `-3` and `-3`
- (d) `-2` and `-4`

**Q27.** What is printed?

```
char c = 'A';
c += 1;
System.out.println(c);
System.out.println(c + 1);
```

- (a) `B` then `B`
- (b) `B` then `67`
- (c) `66` then `67`
- (d) `B` then `C`

**Q28.** After `int age = sc.nextInt();` the user typed `28` and Enter. What does the next `sc.nextLine()` return?

- (a) The next line the user types
- (b) An empty string
- (c) `"28"`
- (d) `null`

**Q29.** Which numeric literal is valid in Java 7 and later?

- (a) `_100`
- (b) `100_`
- (c) `1_000`
- (d) `0x_FF`

**Q30.** Which is true about `StringBuilder` and `StringBuffer`?

- (a) Both are immutable
- (b) `StringBuilder` is thread-safe, `StringBuffer` is not
- (c) `StringBuffer` is thread-safe, `StringBuilder` is faster
- (d) They are identical in every respect

## Part B — Short Answer

**Q31.** Compare local, instance and static variables: where each is declared, where it lives in memory, its default value and its lifetime.

**Q32.** Explain how `float` and `double` are stored. Why is `0.1 + 0.2` not exactly `0.3`, and how should you compare floating-point numbers?

**Q33.** Which values can a floating-point division by zero produce, and how does that differ from integer division by zero? What is special about `NaN`?

**Q34.** Explain widening and narrowing conversions. Why does `byte c = a + b;` fail when `a` and `b` are `byte`, but `c += a` works?

**Q35.** What are the rules for underscores in numeric literals? Which Java version introduced them, and what else was added in the same release for literals?

**Q36.** Differentiate `==` and `equals()` for strings, with an example involving the String pool.

**Q37.** Why is `String` immutable? What are the benefits?

**Q38.** Why should you not use `s += x` in a loop, and what should you use instead? When would you choose `StringBuffer`?

**Q39.** Explain pre-increment vs post-increment, and what `i = i++` does.

**Q40.** Explain switch fall-through. How does the arrow form of switch (Java 14+) differ?

**Q41.** What is the difference between `break`, `continue` and labeled `break`? Why does Java have no `goto`?

**Q42.** Describe upcasting and downcasting, and when `ClassCastException` occurs. How does pattern matching `instanceof` help?

## Part C — Coding Practice

Write each as a small method or `main` program, then compare with the model solutions.

**C1.** Swap two integers without a third variable (two ways).

**C2.** Count the digits of an integer and compute the sum of its digits.

**C3.** Check whether a number is an Armstrong number (for example $153 = 1^3 + 5^3 + 3^3$).

**C4.** Print the first `n` Fibonacci numbers.

**C5.** Find the largest and second-largest element of an array in a single pass, without sorting.

**C6.** Rotate an array to the right by `k` positions, in place.

**C7.** Check whether two strings are anagrams without sorting.

**C8.** Count the frequency of each lowercase letter in a string.

**C9.** Remove duplicates from a sorted array in place and return the new length.

**C10.** Print this pattern for `n = 4`:

```
   *
  ***
 *****
*******
```

**C11.** Compute the sum of both diagonals of a square matrix.

**C12.** Convert a binary string such as `"1101"` to a decimal number without using `Integer.parseInt`.

## Part D — Senior Level

**S1.** For `int[] a = new int[3]; String s = "x"; Person p = new Person();` inside a method, describe exactly what lives on the stack, the heap and the String pool.

**S2.** Your service calculates prices with `double` and customers report 1-paisa differences. Explain the cause, and design the fix. Mention at least three `BigDecimal` pitfalls.

**S3.** Explain the `Integer` cache: its range, how it can be changed, and why `==` on boxed values is dangerous. What is the performance cost of boxing in a hot loop?

**S4.** Predict and explain the output, then say when `intern()` is worth using:

```
String a = "java";
String b = "ja" + "va";
String c = new String("ja") + "va";
final String j = "ja";
System.out.println(a == b);
System.out.println(a == c);
System.out.println(a == c.intern());
System.out.println(a == j + "va");
```

**S5.** What exactly does `javac` generate for string concatenation with `+` in Java 8 vs Java 9+? Is concatenating in a loop still a problem on modern JDKs?

**S6.** A binary search uses `int mid = (lo + hi) / 2;`. Why can this be a latent bug, and how do you fix it? Name two other overflow traps with `int`.

**S7.** Explain why `Math.abs(Integer.MIN_VALUE)` is negative and how two's complement explains the asymmetric range of integers.

**S8.** What is a compile-time constant? Explain why changing a `public static final int` in one library class can leave a dependent jar using the old value, and what the safer pattern is.

**S9.** Explain array covariance and `ArrayStoreException`. Why does Java make generics invariant instead?

**S10.** How is a `switch` on `String` implemented? What happens with `null`? Why does a switch expression have to be exhaustive, and how do sealed types and enums help?

**S11.** Why must local variables used in a lambda be effectively final? What is definite assignment, and how does it relate?

**S12.** How do `Double.compare`, `==` and `equals` treat `NaN` and `-0.0`? What does that mean for sorting a `double[]` or using `Double` as a map key?

**S13.** `Scanner` vs `BufferedReader`: why is one much faster for large inputs, and how would you read one million integers from standard input efficiently?

**S14.** The following compiles. Explain each line's result and the rule behind it:

```
int x = 10;
x += 3.7;                 // ?
byte b = 10;
b = (byte) (b * 30);      // ?
char c = 'a';
c *= 2;                   // ?
long big = Integer.MAX_VALUE + 1;       // ?
long ok  = Integer.MAX_VALUE + 1L;      // ?
```

# Answers — Basic and Senior Level

## Part A — Multiple Choice

| Q | Answer | Explanation |
|---|---|---|
| 1 | (c) | Identifiers may contain letters, digits, `_` and `$`, but cannot start with a digit or contain `-`, and cannot be a keyword. |
| 2 | (c) `true` | `true`, `false` and `null` are literals. `goto` and `const` are reserved keywords; `strictfp` is a keyword. |
| 3 | (b) | `char` is a 16-bit unsigned UTF-16 code unit on every platform. |
| 4 | (c) | Local variables get no default value. Reading one before assignment is a compile error (definite assignment). |
| 5 | (b) `30304050` | Left to right: `10 + 20 = 30`, then `30 + "30"` gives `"3030"`, after which `+ 40` and `+ 50` are concatenations. |
| 6 | (c) -128 | `byte` overflows and wraps; `++` includes an implicit cast. |
| 7 | (b) 16 | A leading `0` makes a literal octal: `010` is 8, so 8 + 8 = 16. |
| 8 | (c) | In `double`, 0.1 + 0.2 is 0.30000000000000004, so the first is `false`. In `float`, the rounded sum happens to equal `0.3f`, so the second is `true`. Neither is safe to rely on. |
| 9 | (c) `Infinity` | Floating-point division by zero does not throw. Only integer division by zero does. |
| 10 | (a) 5.5 | `5/2 = 2`, `5%2 = 1`, `5/2.0 = 2.5`, so 2 + 1 + 2.5. |
| 11 | (b) | `3.14` is a `double`; assigning it to `float` is a lossy conversion. |

| Q | Answer | Explanation |
|---|---|---|
| 12 | (b) | `s += 1` includes an implicit cast. `s + 1` is `int`; `200` does not fit `byte`; `3.5` does not fit `int`. |
| 13 | (c) `false true` | `Integer` caches -128 to 127. 128 creates two objects, so `==` is `false`; 100 is cached, so `true`. |
| 14 | (b) | `new String` is a distinct object, so `a == b` is `false`. `"h" + "i"` is a compile-time constant folded to `"hi"`, so `a == c` is `true`. |
| 15 | (c) 2 | The pooled literal `"abc"` plus the new heap object. |
| 16 | (c) `long` | Allowed: `byte`, `short`, `char`, `int` (and wrappers), `String`, `enum`. |
| 17 | (a) 3 | Post-increment returns the old value 3, increments to 4, then the assignment stores 3 back. |
| 18 | (a) | `'a' + 'b'` is `int` addition (97 + 98 = 195). In `"" + 'a' + 'b'` the string is on the left, so characters are appended. |
| 19 | (c) 12 | `x++` gives 5 (x becomes 6), `++x` gives 7 (x becomes 7): 5 + 7. |
| 20 | (a) | `-16 >> 2` keeps the sign: -4. `-16 >>> 28` fills with zeros: 0xFFFFFFF0 shifted right by 28 leaves 0xF = 15. |
| 21 | (c) | `do-while` tests the condition after the body. |
| 22 | (b) `013` | 0 and 1 print, 2 is skipped by `continue`, 3 prints, 4 triggers `break`. |
| 23 | (c) | Valid indexes are 0 to 2. |
| 24 | (c) `9 false` | `b` is an alias of `a`, so `a[0]` is 9. Array `equals()` is reference equality, so comparing with a new array is `false`. Use `Arrays.equals`. |
| 25 | (a) 2 | `split` removes trailing empty strings: `["a", "b"]`. Use `split(",", -1)` to keep them. |
| 26 | (b) | `Math.round(-2.5)` rounds half toward positive infinity: -2. A cast truncates toward zero: -3. |

| Q | Answer | Explanation |
|---|---|---|
| 27 | (b) | `c += 1` keeps type `char`: `B`. `c + 1` is `int`: 66 + 1 = 67. |
| 28 | (b) | `nextInt()` leaves the line break in the buffer, so `nextLine()` returns the empty remainder of that line. |
| 29 | (c) | Underscores cannot start or end a literal, or follow `0x`. |
| 30 | (c) | `StringBuffer` methods are synchronized; `StringBuilder` skips locking and is faster. Both are mutable. |

## Part B — Short Answer (model answers)

**Q31 — Variable kinds.** Local variables are declared in a method or block, live in the stack frame, have **no default** and die when the block ends. Instance variables are declared in the class without `static`, live in each object on the heap, get defaults (`0`, `false`, `null`) and live as long as the object. Static variables are declared with `static`, exist **once per class**, are created when the class is initialized, get defaults and live as long as the class is loaded.

**Q32 — float and double.** Both use IEEE 754 binary floating-point: sign bit, exponent and mantissa (`float`: 1/8/23 bits, about 7 digits; `double`: 1/11/52 bits, about 15 to 16 digits). Many decimal fractions, such as 0.1, repeat forever in binary and are stored as the nearest representable value, so small errors appear and accumulate: `0.1 + 0.2` is 0.30000000000000004. Therefore compare with a tolerance, `Math.abs(a - b) < 1e-9`, never `==`. For exact decimals such as money use `BigDecimal` (built from a `String`) or integer minor units.

**Q33 — Division by zero.** Floating-point division returns `Infinity`, `-Infinity` or `NaN` (for `0.0 / 0.0`) and never throws. Integer division or remainder by zero throws `ArithmeticException`. `NaN` is unordered: `NaN == NaN` is `false`, and every comparison with it is `false`, except `!=`. Test with `Double.isNaN(x)`. It also propagates through arithmetic.

**Q34 — Widening and narrowing.** Widening goes from a smaller to a larger type (`byte → short → int → long → float → double`, `char → int`) and is automatic. Narrowing needs an explicit cast and can lose data: fractions are truncated, integers keep only low

bits. In `a + b`, `byte` operands are promoted to `int`, so the result is `int` and cannot be assigned to `byte` without a cast. A compound assignment such as `c += a` expands to `c = (byte)(c + a)`, so the cast is inserted for you.

**Q35 — Underscores in literals.** Java 7 allowed underscores between digits to improve readability: `1_000_000`, `0b1010_1010`, `0xFF_EC`. Not allowed: at the start or end, next to the decimal point, before `L`, `F` or `D`, or right after `0x` or `0b`. Java 7 also added **binary literals** (`0b1010`).

**Q36 — `==` vs `equals()`.** `==` compares references; `equals()` compares characters. Literals are pooled, so `"hi" == "hi"` is `true`, but `new String("hi") == "hi"` is `false` even though `equals` is `true`. Always use `equals` (or `equalsIgnoreCase`), writing the literal first to avoid a `NullPointerException`.

**Q37 — Why immutable.** Security (values such as paths, class names and credentials cannot change after checks), thread safety without locks, cached hash codes (good map keys), and safe sharing through the String pool, which saves memory.

**Q38 — Concatenation in loops.** Each `s += x` allocates a new string and copies all previous characters, so the cost grows quadratically with the number of iterations. Use `StringBuilder`. Use `StringBuffer` only when several threads must append to the same builder; usually it is better to give each thread its own builder.

**Q39 — Increment.** `++i` increments, then yields the new value. `i++` yields the old value, then increments. In `i = i++` the old value is saved, `i` is incremented, and then the saved old value is assigned back, so `i` is unchanged.

**Q40 — Switch.** In the classic switch, execution continues into following `case` blocks until a `break`, `return` or `throw`. That is useful for grouping labels but is a common bug. The arrow form (`case X ->`) runs only its own branch, accepts several labels (`case 1, 2 ->`), and as an expression returns a value (with `yield` inside a block). A switch expression must cover all cases.

**Q41 — Jumps.** `break` leaves the nearest loop or switch. `continue` skips to the next iteration of the nearest loop. A labeled `break` / `continue` targets a named outer loop. Java omits `goto` because arbitrary jumps make code hard to reason about; labeled jumps, `return`, loops and exceptions cover the real needs. `goto` remains a reserved word.

**Q42 — Reference casts.** Upcasting (subclass reference to superclass type) is implicit and always safe. Downcasting needs an explicit cast and is checked at runtime: if the object is not actually an instance of the target type, `ClassCastException` is thrown.

Guard with `instanceof`; pattern matching (`if (a instanceof Dog d)`) tests, casts and declares the variable in one step.

## Part C — Coding Practice (model solutions)

### C1. Swap without a third variable

```
// Arithmetic: works even if a + b overflows, because int arithmetic wraps
consistently
a = a + b;
b = a - b;
a = a - b;

// XOR
a ^= b;
b ^= a;
a ^= b;
```

In real code use a temporary variable; it is clearer and just as fast. Note the XOR swap fails if both names refer to the same array slot.

### C2. Digit count and digit sum

```
long n = Math.abs((long) num);        // long avoids the
Math.abs(Integer.MIN_VALUE) trap
int digits = (n == 0) ? 1 : 0, sum = 0;
for (long t = n; t > 0; t /= 10) {
    digits++;
    sum += (int) (t % 10);
}
```

### C3. Armstrong number

```
static boolean isArmstrong(int n) {
    int d = String.valueOf(n).length();
    int sum = 0;
    for (int t = n; t > 0; t /= 10) {
        int digit = t % 10, p = 1;
        for (int i = 0; i < d; i++) p *= digit;
        sum += p;
    }
    return sum == n;              // 153, 370, 371, 407, 1634 ...
}
```

### C4. Fibonacci

```
long a = 0, b = 1;
for (int i = 0; i < n; i++) {
    System.out.print(a + " ");
    long next = a + b;
    a = b;
    b = next;                      // long overflows after the 92nd term
}
```

### C5. Largest and second largest in one pass

```
int max = Integer.MIN_VALUE, second = Integer.MIN_VALUE;
for (int v : a) {
    if (v > max)                  { second = max; max = v; }
    else if (v > second && v != max) { second = v; }
}
```

Edge cases: if all elements are equal there is no second largest, so `second` stays at the sentinel. For arrays that may contain `Integer.MIN_VALUE`, track presence with `boolean` flags or use `Integer` objects and `null`.

### C6. Rotate right by k, in place (three reversals, O(n) time, O(1) space)

```
static void rotate(int[] a, int k) {
    int n = a.length;
    if (n == 0) return;
    k = ((k % n) + n) % n;         // handles k larger than n and negative k
    reverse(a, 0, n - 1);
    reverse(a, 0, k - 1);
    reverse(a, k, n - 1);
}
static void reverse(int[] a, int i, int j) {
    while (i < j) { int t = a[i]; a[i++] = a[j]; a[j--] = t; }
}
```

### C7. Anagram check without sorting (counting)

```
static boolean anagram(String s, String t) {
    if (s.length() != t.length()) return false;
    int[] count = new int[26];     // lowercase a-z; use new int[256] for
ASCII
    for (int i = 0; i < s.length(); i++) {
        count[s.charAt(i) - 'a']++;
        count[t.charAt(i) - 'a']--;
    }
    for (int c : count) if (c != 0) return false;
    return true;
}
```

### C8. Letter frequency

```
int[] freq = new int[26];
for (char c : s.toCharArray()) {
    if (c >= 'a' && c <= 'z') freq[c - 'a']++;
}
for (int i = 0; i < 26; i++) {
    if (freq[i] > 0) System.out.println((char) ('a' + i) + " = " + freq[i]);
}
```

### C9. Remove duplicates from a sorted array (two pointers)

```
static int dedupe(int[] a) {
    if (a.length == 0) return 0;
    int w = 1;                          // next write position
    for (int r = 1; r < a.length; r++) {
        if (a[r] != a[w - 1]) a[w++] = a[r];
    }
    return w;                           // first w elements are the unique
values
}
```

### C10. Pyramid

```
for (int i = 1; i <= n; i++) {
    System.out.println(" ".repeat(n - i) + "*".repeat(2 * i - 1));
}
```

### C11. Sum of both diagonals

```
int sum = 0;
for (int i = 0; i < n; i++) {
    sum += m[i][i] + m[i][n - 1 - i];
}
if (n % 2 == 1) sum -= m[n / 2][n / 2];   // centre was counted twice
```

### C12. Binary string to decimal

```
static int toDecimal(String bits) {
    int result = 0;
    for (int i = 0; i < bits.length(); i++) {
        char c = bits.charAt(i);
        if (c != '0' && c != '1') throw new NumberFormatException("Not
binary: " + bits);
        result = result * 2 + (c - '0');   // or: (result << 1) | (c - '0')
    }
    return result;                          // "1101" -> 13 (overflows past 31
bits)
}
```

## Part D — Senior Level (model answers)

**S1 — Memory layout.** In the method's stack frame there are three slots holding **references**: `a`, `s` and `p`. On the heap: the `int[3]` object (object header, length, and 12 bytes of zeroed data) and the `Person` object with its fields. The literal `"x"` is a `String` object in the **String pool** (the JVM's string table, which holds heap objects); `s` references it. Primitives declared as locals live directly in the frame. If the JIT proves that `a` or `p` does not escape the method, escape analysis may remove the allocation entirely (scalar replacement), so this picture is the semantic model, not a promise about the machine code.

**S2 — Money with `double`.** Prices such as 19.99 are not representable in binary, so repeated multiplication, tax and discount steps accumulate representation error and different summation orders give different rounding. Fix: use `BigDecimal` for all money maths (or store **minor units** as `long`), pick an explicit rounding policy (`HALF_UP` or `HALF_EVEN` as the business requires) and apply it at defined points, store as `NUMERIC(19,4)` or similar in the database, and serialize as a string or exact decimal in JSON. `BigDecimal` pitfalls: (1) `new BigDecimal(0.1)` captures the double's error; use `new BigDecimal("0.1")` or `BigDecimal.valueOf(0.1)`. (2) `equals` compares scale too (`1.0` is not equal to `1.00`) and so affects `HashMap` keys and sets; use `compareTo` for

numeric equality. (3) `divide` without a scale or `MathContext` throws on non-terminating results such as 1/3. (4) It is immutable: `a.add(b);` alone does nothing. (5) `toString` may print scientific notation after `stripTrailingZeros`; use `toPlainString()`. (6) `setScale` changes decimal places while `round(MathContext)` changes significant digits.

**S3 — Integer cache.** `Integer.valueOf` (used by autoboxing) returns cached instances for -128 to 127; `Short`, `Byte`, `Long` and `Character` (0 to 127) have similar caches, and `Boolean` has two constants. The upper bound of the `Integer` cache can be raised with `-XX:AutoBoxCacheMax=<n>`; the lower bound is fixed. `==` on boxed values therefore works for small numbers by accident and fails for larger ones; always use `equals` or unbox. Boxing costs an allocation per value outside the cache, extra memory (an `Integer` is about 16 bytes versus 4), pointer chasing and GC pressure, and unboxing a `null` throws `NullPointerException`. Prefer primitives and primitive streams (`IntStream`) in hot paths; do not rely on escape analysis to remove boxing.

**S4 — Output: `true`, `false`, `true`, `true`.** `"ja" + "va"` is a compile-time constant folded to `"java"`, so `b` is the pooled instance. `new String("ja") + "va"` is built at runtime, producing a new heap object, so `a == c` is `false`. `c.intern()` returns the canonical pooled string, which is `a`'s object, so `true`. `j` is a `final String` initialized with a constant, so `j + "va"` is also a constant expression: `true`. `intern()` is worth it only for large numbers of long-lived duplicate strings (for example repeated tokens from a huge input) where memory savings outweigh the string-table lookup cost (`-XX:StringTableSize`); alternatives are your own deduplication map or G1's `-XX:+UseStringDeduplication`. Since Java 7 interned strings are ordinary heap objects and can be garbage collected.

**S5 — Concatenation bytecode.** Before Java 9, `javac` rewrote `a + b + c` into `new StringBuilder().append(a).append(b).append(c).toString()`. Since Java 9 (JEP 280) it emits one `invokedynamic` call bound at runtime to `StringConcatFactory`, which can size the result exactly and avoid the intermediate builder, and can be improved by the JDK without recompiling your code. In a loop it is still a problem on any JDK: each iteration concatenates the accumulated string with the new part, creating a new string and copying all earlier characters, so total work is quadratic. The JIT does not merge work across iterations. Use one `StringBuilder` outside the loop.

**S6 — The midpoint bug.** `lo + hi` can exceed `Integer.MAX_VALUE` when the array has more than about 2^30 elements; the sum wraps to a negative number and `mid` becomes negative, causing an `ArrayIndexOutOfBoundsException`. This bug lived in the JDK's own binary search for years. Fix: `int mid = lo + (hi - lo) / 2;` or `(lo + hi) >>> 1`.

Other `int` traps: `Math.abs(Integer.MIN_VALUE)` stays negative; `long total = a * b;` multiplies as `int` first, then widens; `i <= Integer.MAX_VALUE` in a loop never ends; `hash % n` can be negative (use `Math.floorMod`); and `(int) someLong` silently drops high bits. Use `Math.addExact`, `multiplyExact` and `toIntExact` to fail fast.

**S7 — `Math.abs(Integer.MIN_VALUE)`.** Signed integers use two's complement. With 32 bits there are 2^32 patterns: one is zero, 2^31 - 1 are positive, and 2^31 are negative, so the range is -2^31 to 2^31 - 1 and has no positive twin for -2^31. Negation inverts the bits and adds one; for `0x80000000` that gives `0x80000000` again, so `-MIN_VALUE == MIN_VALUE` and `Math.abs` returns a negative number. `Math.negateExact` and `Math.absExact` (Java 15) throw `ArithmeticException` instead.

**S8 — Compile-time constants.** A `static final` primitive or `String` initialized with a constant expression is inlined by `javac` into every class that uses it. If a library later changes `public static final int MAX = 5` to `10` and is redeployed without recompiling its dependents, those dependents keep using `5`, because the value is stored in their own constant pool. Safer patterns: use constants only for values that truly never change (`Integer.MAX_VALUE`, mathematical constants); otherwise expose a method (`max()`), an `enum`, configuration, or a non-constant initializer such as `static final int MAX = Integer.valueOf(5)` so the value is read at runtime. Avoid putting constants in interfaces just to inherit them.

**S9 — Array covariance.** Arrays are covariant: `String[]` is a subtype of `Object[]`. That allowed generic algorithms such as `Arrays.sort(Object[])` before generics existed, but it is unsound: `Object[] o = new String[1]; o[0] = 1;` compiles and fails at runtime with `ArrayStoreException`, because arrays carry their element type (they are **reified**) and check every store. Generics are **invariant** (`List<String>` is not a `List<Object>`) and are erased at runtime, so the compiler must prevent the problem statically. Flexibility is restored with wildcards (`List<? extends Number>` for reading, `List<? super Integer>` for writing). For the same reason you cannot write `new T[n]` or `new List<String>[n]`.

**S10 — Switch internals.** A switch on `String` compiles to a switch on `hashCode()` of the selector, followed by `equals` checks to resolve collisions and then a second switch on the matched index. A `null` selector throws `NullPointerException` (Java 21 allows `case null`). A switch **expression** must produce a value for every possible input, so the compiler requires exhaustiveness: a `default`, or every enum constant, or every permitted subtype of a sealed type. The benefit is evolution safety: add a constant or a

subtype and every switch without a `default` fails to compile, pointing to code that needs updating. Without recompilation, the compiler-inserted fallback throws at runtime (`MatchException` for pattern switches in Java 21).

**S11 — Effectively final.** A lambda or inner class captures the **value** of a local variable, because the frame that owns the variable may be gone, or on another thread, when the lambda later runs. If the variable could change after capture, there would be two diverging copies, or shared mutable state across threads without synchronization. Requiring the variable to be `final` or effectively final (never reassigned) makes copy and original always agree. **Definite assignment** is the compiler's flow analysis proving a variable has been assigned on every path before it is read; a captured variable must be definitely assigned before the lambda body. Fields are exempt because the lambda captures `this`, not the field value. Workarounds such as a one-element array or `AtomicInteger` work but deserve thought about thread safety; often a stream `reduce` or a loop is cleaner.

**S12 — `NaN` and `-0.0`.** With primitives, `NaN == NaN` is `false` and `0.0 == -0.0` is `true`. `Double.equals` and `Double.compare` use the raw bit patterns and define a **total order**: `NaN` equals itself and sorts above every number, and `-0.0` sorts below `0.0`. `Arrays.sort(double[])` uses that total order, so `-0.0` comes before `0.0` and `NaN` ends up last. A hand-written comparator that uses `<` and `>` on raw doubles breaks transitivity when `NaN` is present, and `TimSort` can then throw `IllegalArgumentException: Comparison method violates its general contract`. As a `HashMap` key, `Double` follows `equals`, so `NaN` works as a key and `0.0` and `-0.0` are two different keys. Floating-point keys are best avoided.

**S13 — Fast input.** `Scanner` tokenizes with regular expressions, creates a `String` per token and does extra bookkeeping on every call, so it is often five to ten times slower than reading raw buffers. `BufferedReader` reads large chunks into a buffer; combined with `StreamTokenizer` or a manual digit parser it avoids per-token overhead. For one million integers:

```
StreamTokenizer in = new StreamTokenizer(new BufferedReader(new
InputStreamReader(System.in)));
in.nextToken();
int n = (int) in.nval;                // numbers are parsed as double, exact
for ints
long sum = 0;
for (int i = 0; i < n; i++) { in.nextToken(); sum += (long) in.nval; }
```

For output, collect results in a `StringBuilder` or use a buffered `PrintWriter` and flush once, instead of calling `System.out.println` a million times.

**S14 — Compound assignments and promotion.**

```
int x = 10;
x += 3.7;                 // 13: x = (int)(10 + 3.7); the hidden cast
truncates 13.7
byte b = 10;
b = (byte) (b * 30);      // 44: 300 does not fit in a byte, so 300 - 256 =
44
char c = 'a';
c *= 2;                   // 'Â' (U+00C2): (char)(97 * 2 = 194)
long big = Integer.MAX_VALUE + 1;       // -2147483648: the addition is done
in int and overflows first
long ok  = Integer.MAX_VALUE + 1L;      // 2147483648: one long operand makes
the whole addition long
```

The rule behind all of them: the **type of an expression is decided by its operands**, not by the variable it will be assigned to, and a compound assignment inserts a narrowing cast silently.

---

**End of Section 2.** The Basics track is complete. Next could be Object-Oriented Programming (classes, objects, constructors, `static`, inheritance, polymorphism, abstraction, interfaces, encapsulation), or a revision quiz on Sections 1 and 2.